EHMbrAIn

AI-based versus traditional Engine Health Monitoring:
a gas path analysis testbed on the CFM56-7B with synthetic ground truth

— (anonymous in the public repository) —

July 4, 2026

Living document: updated at every project milestone.

Contents

Notation and acronyms **4**

1 Introduction **5**
1.1 Motivation . . . 5
1.2 Objective . . . 5
1.3 Hypotheses . . . 6
1.4 Contributions . . . 7
1.5 Document structure . . . 7

2 Background **8**
2.1 How a two-spool turbofan works . . . 8
2.2 The subject engine: CFM56-7B26 . . . 8
2.3 What is measured, and why parameters must be corrected . . . 9
2.4 Gas Path Analysis . . . 10
2.4.1 Health parameters . . . 10
2.4.2 The linear model . . . 10
2.4.3 Estimation, and why it is hard . . . 11
2.4.4 Smearing: a two-parameter toy example . . . 11
2.4.5 Smearing in general . . . 11
2.5 The physics of degradation . . . 12

3 Methodology and testbed architecture **13**
3.1 Overall architecture . . . 13
3.2 Machine learning without prior background . . . 14
3.3 The machine-learning toolbox, briefly . . . 14
3.4 Methodological safeguards . . . 15
3.5 Reproducibility and infrastructure . . . 16
3.6 Software architecture and computational pipeline . . . 16
3.6.1 Library modules (`src/ehmbrain/`) . . . 16
3.6.2 Executable scripts (`scripts/`) . . . 17
3.6.3 Test suite (`tests/`) . . . 18
3.7 Replication guide . . . 18
3.7.1 Compute cost on the reference machine . . . 19
3.8 Comparison metrics . . . 20

4 Thermodynamic twin of the CFM56-7B26 **21**
4.1 What a cycle model is . . . 21
4.2 Strategy: adapt, don't build from scratch . . . 21
4.3 Design point (WP1.1) . . . 22
4.4 Calibration against measured data (WP1.2) . . . 23
4.4.1 Data sources and philosophy . . . 23
4.4.2 Calibration iterations . . . 23

4.4.3 Idle convergence: diagnosis and cure . . . 24
4.5 Baseline decks (WP1.3) . . . 25
4.5.1 Corrected-parameter exponents and the corrected-space baseline . . . 26
4.6 Influence coefficient matrix and observability (WP1.4) . . . 27
4.6.1 Health-parameter injection . . . 27
4.6.2 The ICM at the reference conditions . . . 27
4.6.3 Observability: the quantitative case for hypothesis H2 . . . 27
4.6.4 Thermodynamics cross-check . . . 28
4.7 Declared limitations . . . 28
4.8 Automated verification . . . 29

5 The synthetic fleet: SynCFM56 30
5.1 Generator architecture: linearize, then audit . . . 30
5.2 Ground-truth trajectories . . . 30
5.3 Snapshots and the measurement model . . . 31
5.4 Gate audits — including the one that failed . . . 33
5.4.1 Nonlinearity of the generator . . . 33
5.4.2 Difficulty: the audit that failed, and what it taught . . . 33
5.4.3 Realism . . . 34
5.4.4 Version 1.1: more episodes, and the gate bites again . . . 34
5.5 Milestone H2: declared closed . . . 34

6 The traditional EHM pipeline 35
6.1 Pipeline overview . . . 35
6.2 Two detector-design lessons . . . 35
6.3 Results on the test fleet (v0 defaults) . . . 36
6.4 Milestone H3 . . . 38

7 The AI EHM suite 39
7.1 Fairness by construction . . . 39
7.2 Prognosis: the headline result . . . 39
7.3 Detection and diagnosis: two instructive failures . . . 40
7.4 The hybrid experiment: a negative result worth having . . . 41
7.5 The Physics-Consistency Score: machinery delivered, null result at v0 . . . 41
7.6 Detection: a design gap, not a threshold gap . . . 42
7.7 Status toward milestone H4 . . . 42

8 Confirmatory results: the pre-registered verdicts 43
8.1 Protocol executed as frozen . . . 43
8.2 H1 confirmed: the story is the tuning, not the model . . . 44
8.3 H2 refuted — the observability wall binds everyone . . . 44
8.4 H3 and H5 confirmed — prognosis is where the AI pays . . . 44
8.5 H4 refuted — physics features are not free wins . . . 44
8.6 What an engineering organization should take from this . . . 44
8.7 Sim-to-real: the ranking survives on the canonical external benchmark . . . 45

9 Case studies: five engines, told in operations language 46

10 Conclusions and future work 49
10.1 What was built . . . 49
10.2 What the evidence says . . . 49
10.3 Guidance for adoption . . . 49
10.4 Honest limitations . . . 50

10.5 Future work 50

A Decision register **51**

B Milestone log **54**

References **56**

Notation and acronyms

Acronyms

BPR	bypass ratio	LOO	leave-one-out (cross-validation)
CEA	NASA Chemical Equilibrium with Applications	LPC	low-pressure compressor (booster)
CI	continuous integration	LPT	low-pressure turbine
C-MAPSS	NASA turbofan simulation benchmark	MAE	mean absolute error
EEDB	ICAO Engine Emissions Databank	OPR	overall pressure ratio
EGT(M)	exhaust gas temperature (margin)	PC	power code: fraction of reference thrust
EHM	engine health monitoring	PCS	Physics-Consistency Score (ours, C5)
FADEC	full-authority digital engine control	PHM	prognostics and health management
FAR	fuel–air ratio	RMSE	root-mean-square error
FOD	foreign-object damage	RUL	remaining useful life
FPR	false-positive rate	SAC	single annular combustor
GPA	gas path analysis	SHAP	Shapley additive explanations
HPC	high-pressure compressor	SLS	sea-level static
HPT	high-pressure turbine	SVD	singular value decomposition
ICM	influence coefficient matrix	TCDS	type-certificate data sheet
ISA	international standard atmosphere	TSFC	thrust-specific fuel consumption
N1, N2	low-/high-pressure spool speeds	WLS	weighted least squares

Symbols

$\theta,\ \delta$	inlet total temperature / pressure ratios to ISA sea level	$\mathbf{H}$	influence coefficient matrix
$\Delta\eta$	component efficiency deviation [%]	$\mathbf{Q},\ \mathbf{R}$	process / measurement noise covariance
$\Delta\Gamma$	component flow-capacity deviation [%]	$\mathbf{P}_0,\ \lambda$	prior shape / strength of the WLS regularization
$\mathbf{x}$	health-parameter vector (10 components)	$N1_c$	corrected fan speed $N1/\sqrt{\theta}$
$\Delta\mathbf{z}$	measurement deviations from baseline	T_{t4}	combustor-exit total temperature
$W_f,\ F_n$	fuel flow, net thrust	ΔT_{ISA} (dTs)	ambient offset from the ISA day

Reading order. Every technical term is defined at first use; Chapter 2 builds the engine and GPA foundations, Section 3.3 the machine-learning toolbox. The hypotheses table (Table 1.1) can be read superficially on a first pass and revisited after those two stops.

Chapter 1

Introduction

This chapter in brief: jet engines degrade invisibly, airlines monitor them through a handful of measurements, and two schools of methods compete to interpret those measurements. This project builds a level playing field to compare them. If you are new to the field, skim the hypotheses table on first read and return to it after Chapter 2.

1.1 Motivation

A commercial jet engine degrades from its very first flight. Dust and pollution stick to compressor blades, hot-section parts creep and oxidize, tip clearances open up, seals wear. The airline cannot see any of this directly —the engine stays on the wing for years— so it watches the engine's *symptoms* instead: the temperatures, pressures, shaft speeds and fuel flow recorded on every flight. The discipline that turns those symptoms into maintenance decisions is called *Engine Health Monitoring* (EHM). The stakes are large: a single shop visit for an engine overhaul costs millions of dollars, and an unscheduled engine removal disrupts an entire fleet schedule.

The classical analytical core of EHM, in use since the 1970s, is *Gas Path Analysis* (GPA) [1]. The idea: each physical fault (a dirty compressor, an eroded turbine) leaves a characteristic fingerprint on the measured parameters; by inverting a thermodynamic model of the engine, one can estimate *which* component has deteriorated and *by how much* from the measurements alone. Around GPA the industry built a mature toolbox —baseline models, exponential trend smoothing, Kalman filters, expert fault-isolation rules [2]— which this project collectively calls *traditional EHM*.

Over the last decade, machine learning has produced striking results in the adjacent research field of prognostics: predicting the *Remaining Useful Life* (RUL) of an engine, i.e. how many flights remain before it must come off the wing, mostly on synthetic benchmark datasets such as NASA's C-MAPSS [3] and N-CMAPSS [4]. Yet the literature has a structural weakness: **AI methods are almost never compared against a traditional pipeline implemented with equal care**, on the same data, with the same metrics and the same tuning effort. The result is a stream of comparisons against straw men, which makes it impossible to know what AI actually contributes, under which conditions, and at what cost.

1.2 Objective

This project builds a reproducible testbed in which both EHM families —traditional and AI-based— observe *exactly the same data* from a synthetic fleet of CFM56-7B26 turbofans and are scored with *exactly the same metrics*. The methodological key is that the synthetic data carries **complete ground truth**: the true health state of every engine component (its efficiency and flow-capacity deviations, defined in Section 2.4) at every flight. With real airline data this is

impossible —nobody knows the true internal state of an engine on the wing— but with synthetic data one can measure not only whether a method detects or predicts well, but whether it *attributes the damage to the correct component*. That attribution question is precisely where traditional GPA is weakest (the *smearing* phenomenon, Section 2.4) and where AI is least transparent (the black-box problem).

Audience and register. This document is a research work aimed at two readers at once: the engineering organization deciding whether and how to integrate AI into its engine health monitoring practice, and the researcher looking for a rigorous, reproducible benchmark at the knowledge frontier. That dual audience sets the register: every method is pushed to publishable rigor (pre-registration, statistical tests, audited datasets), and every concept is introduced without assuming prior specialization — chapter briefs, worked numeric examples and a notation front-matter keep the document self-contained. Advancing the frontier and being readable are treated as the same job, not competing ones.

Since neither a performance model of the engine nor degradation data were available to the project, both are built from scratch: a thermodynamic "digital twin" of the CFM56-7B26 in pyCycle [5], calibrated against certified public data (Chapter 4), and a fleet degradation generator with physically derived fault signatures (Chapter 5).

1.3 Hypotheses

Each hypothesis is formalized with quantitative thresholds *before* any test-set result is seen (the pre-registration protocol of Section 3.4). All are tested with a Wilcoxon signed-rank test paired by engine, Holm correction for multiple comparisons ($\alpha = 0.05$), and BCa bootstrap confidence intervals. In plain words: for every comparison we check that the improvement is statistically significant, not a fluke of one lucky engine. The table below necessarily uses terms defined later —the GPA quantities in Chapter 2, the machine-learning methods in Section 3.3, every metric in the metrics section of Chapter 3, and all acronyms in the notation chapter— so it is meant to be skimmed now and revisited after those stops.

Table 1.1: Project hypotheses and their confirmation criteria.

ID	Hypothesis	Confirmation criterion
H1	AI detects faults earlier and with fewer false alarms than trend monitoring	median lead time $\geq +20\,\%$ and false-positive rate $\leq 0.5\times$ versus trending, at a fixed recall of 0.9; $p < 0.05$
H2	AI isolates faults with overlapping signatures better	$+10$ percentage points of isolation accuracy on the "confusable" subset (fault signatures less than 15° apart in the influence-matrix space) versus weighted-least-squares GPA; $p < 0.05$
H3	AI predicts RUL better	simultaneous improvement in RMSE and in the asymmetric NASA PHM08 score; $p < 0.05$
H4	The physics-informed hybrid dominates when data is scarce	RUL RMSE $\leq$ pure AI with $100\,\%$ of the training data and strictly better with $10\,\%$ and $25\,\%$; $p < 0.05$
H5	Conformal intervals are better calibrated than the Kalman covariance	smaller $\lvert$empirical coverage $- 0.9\rvert$ with mean interval width no worse than $+20\,\%$

Refuting any hypothesis is itself a publishable result: pre-registration is what makes the refutation credible. Reformulating hypotheses after the repository tag `prereg-v1` is forbidden.

1.4 Contributions

C1. **SynCFM56**: the first public synthetic GPA dataset for a specific commercial engine (CFM56-7B26), with per-flight health-parameter ground truth and an ACARS-style snapshot format (one takeoff report and one cruise report per flight, the way real airline EHM data actually arrives), more realistic than the continuous time series of C-MAPSS.

C2. A head-to-head comparison under a **pre-registered protocol** with a symmetric tuning budget (the same number of Optuna trials) for both families.

C3. A physics-informed hybrid with three separately ablated mechanisms: residual features against the digital twin, physically constrained loss functions, and GPA→ML stacking.

C4. Compared uncertainty quantification: conformal-prediction RUL intervals versus the Kalman filter covariance.

C5. The **Physics-Consistency Score (PCS)**: a new explainability metric that projects SHAP attributions into the physical health-parameter space through the influence coefficient matrix.

C6. A determinability map: a sensors × noise × data-size ablation showing *when* each approach wins.

C7. Sim-to-real validation: the full methodology is re-run on N-CMAPSS to check that the method ranking is not an artifact of our own simulator.

1.5 Document structure

Chapter 2 builds the technical foundations assuming no prior knowledge of jet engines or GPA: how a turbofan works, what is measured on the wing, and the mathematics of gas path diagnosis. Chapter 3 describes the testbed architecture and the methodological safeguards. Chapter 4 documents the thermodynamic twin of the CFM56-7B26 and its calibration against certified data —the work completed to date— with quantitative evidence of every step. **??** summarizes the remaining phases and their acceptance criteria.

Chapter 2

Background

This chapter in brief: how a turbofan works and what its two spools are; which handful of numbers an airline actually sees; why raw readings must be corrected before comparison (with a worked example); how gas path analysis turns corrected deviations into a health diagnosis; a two-line toy problem showing exactly how that diagnosis fails (smearing); and how real engines physically degrade. Everything later builds on these six ideas.

This chapter assumes no prior knowledge of gas turbines. It introduces the engine, the measurements available in service, the mathematics of gas path diagnosis, and the physics of degradation —everything needed to follow the rest of the document.

2.1 How a two-spool turbofan works

A turbofan produces thrust by accelerating air backwards. Air enters through the *fan* (the large visible rotor); most of it —the *bypass* stream— goes around the core and exits through the bypass nozzle, producing the majority of the thrust. The rest enters the *core*: it is compressed by a low-pressure compressor (the *booster*) and a high-pressure compressor (HPC), heated by burning fuel in the *combustor*, and expanded through a high-pressure turbine (HPT) and a low-pressure turbine (LPT) that extract the work needed to drive the compressors, before leaving through the core nozzle.

The machine is arranged in two mechanically independent *spools* (concentric shafts):

- the **low-pressure (LP) spool**: fan + booster driven by the LPT, rotating at speed **N1**;
- the **high-pressure (HP) spool**: HPC driven by the HPT, rotating at speed **N2**.

The ratio of bypass to core mass flow is the *bypass ratio* (BPR); the ratio of compressor exit pressure to inlet pressure is the *overall pressure ratio* (OPR). Higher BPR and OPR generally mean better fuel efficiency, measured as *thrust-specific fuel consumption* (TSFC): fuel flow per unit of thrust. Figure 2.1 shows the layout and the station numbers used throughout this document.

2.2 The subject engine: CFM56-7B26

The CFM56-7B (CFM International, a Safran–GE joint venture) powers the Boeing 737NG family and is one of the most widely operated commercial engines in history, which guarantees abundant public type data. The **-7B26** variant (117 kN takeoff thrust, Boeing 737-800) is chosen as the most common one. Its type-certificate data sheet (TCDS EASA E.004, issue 07 [6]) describes: a single-stage fan, 3-stage booster, 9-stage HPC, annular combustor, single-stage HPT

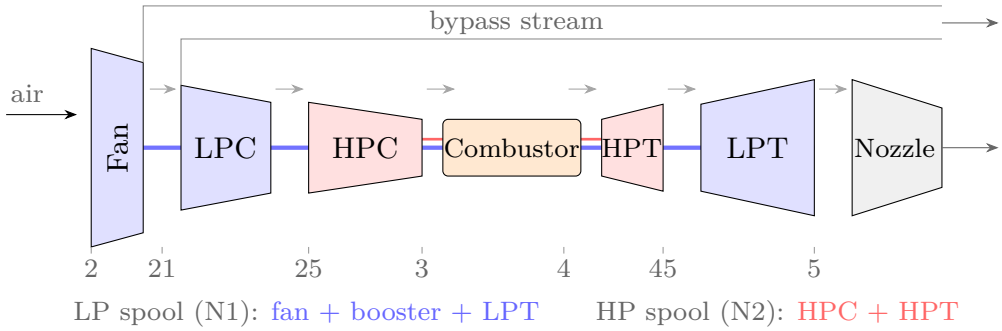

Figure 2.1: Two-spool separate-flow turbofan layout with the SAE station numbers used in this document. The EGT probe of the real CFM56-7B sits inside the LPT (station 49.5); this model uses station 45 (HPT exit) as its consistent EGT proxy.

and 4-stage LPT, governed by a dual-channel FADEC (a full-authority digital engine control —the computer that meters fuel). The pilot-facing thrust-setting parameter is N1, not engine pressure ratio.

Table 2.1: Certified CFM56-7B26 data used as the calibration contract. Sources: TCDS EASA E.004 issue 07 [6] and the ICAO Engine Emissions Databank, edition 03/2026, UID 3CM033 [7].

Quantity	Value	Source	Status
Takeoff thrust	11 699 daN (= 116.99 kN)	TCDS	verified
Maximum continuous thrust	11 521 daN	TCDS	verified
N1 redline (104 %)	5382 rpm ⇒ 100 % = 5175 rpm	TCDS	verified
N2 redline (105 %)	15 183 rpm ⇒ 100 % = 14 460 rpm	TCDS	verified
Max. EGT, takeoff / continuous	950 °C / 925 °C (displayed)	TCDS	verified
Dry weight	2386 kg	TCDS	verified
Bypass ratio	5.1	EEDB	verified
Pressure ratio (sea-level static, rated)	27.61	EEDB	verified
Fuel flow at 100/85/30/7 % F_{00}	1.221 / 0.999 / 0.338 / 0.113 kg s^{-1}	EEDB	verified

Two subtleties from the TCDS matter for modeling and are frequently overlooked:

- The certified *exhaust gas temperature* (EGT) —the single most important health indicator, because it rises as the engine deteriorates— is measured at station **T49.5** (the stage-2 LPT nozzle), *not* at the HPT exit. Moreover, the *displayed* cockpit value adds a "shunt" function of +30 °C (for single-annular-combustor engines, active above 8500 rpm N2) plus a model-dependent "trim" (zero for the -7B26).
- Air bleed limits (air extracted from the compressor for the aircraft's cabin and anti-ice systems) are tabulated per extraction stage and shaft speed; at rated speed the 9th-stage bleed is capped at 7 % of core flow.

2.3 What is measured, and why parameters must be corrected

Engine locations are numbered per the SAE ARP755A standard [8]: station 2 is the fan inlet, 21 the fan exit, 25 the HPC inlet, 3 the HPC exit, 4 the combustor exit, 45 the HPT exit, and 5 the LPT exit. The measurements available in airline service (the "cockpit set") are N1, N2, EGT and fuel flow W_f; some installations add pressures and temperatures at stations 25 and 3.

A raw measurement is meaningless on its own because it depends on the day: the same healthy engine reads a higher EGT on a hot day at a high-altitude airport than on a cold day at

sea level. To compare flights, measurements are *corrected* to standard sea-level conditions using $\theta = T_{t2}/288.15\,\mathrm{K}$ and $\delta = P_{t2}/101.325\,\mathrm{kPa}$ (the ratios of inlet total temperature and pressure to their standard-day values):

$$N_{\mathrm{corr}} = \frac{N}{\sqrt{\theta}}, \qquad W_{f,\mathrm{corr}} = \frac{W_f}{\delta\,\theta^{a}}, \qquad \mathrm{EGT}_{\mathrm{corr}} = \frac{\mathrm{EGT}}{\theta}. \tag{2.1}$$

The exponent a (typically 0.5–0.7) is often copied from generic tables; here it will instead be fitted by regression on sweeps of our own thermodynamic model, removing a classical source of systematic baseline error.

Worked example. A healthy engine reads 880 K of EGT on a standard day ($\theta = 1$). On an ISA+30 day the inlet runs at 318 K, so $\theta = 318/288.15 = 1.104$, and the same healthy engine reads 966 K. A naive comparison would report an "86 K deterioration" where none exists. Corrected with the fitted exponent $a = 0.95$: $966/1.104^{0.95} = 879\,\mathrm{K}$ — the hot-day reading collapses onto the standard-day one, and only *genuine* health changes remain visible. This is why every deviation in this project is computed in corrected space.

2.4 Gas Path Analysis

2.4.1 Health parameters

GPA describes the internal state of the engine with two numbers per turbomachine: the deviation of its *isentropic efficiency* $\Delta\eta$ (how much of the ideal work it actually achieves —dirt and wear reduce it) and the deviation of its *flow capacity* $\Delta\Gamma$ (how much corrected flow it swallows at a given speed —fouling reduces it in compressors; turbine erosion, which opens the effective nozzle area, increases it). With five turbomachines (fan, booster, HPC, HPT, LPT) the health state is the 10-component vector

$$\mathbf{x} = [\Delta\eta_{\mathrm{fan}}, \Delta\Gamma_{\mathrm{fan}}, \Delta\eta_{\mathrm{LPC}}, \Delta\Gamma_{\mathrm{LPC}}, \Delta\eta_{\mathrm{HPC}}, \Delta\Gamma_{\mathrm{HPC}}, \Delta\eta_{\mathrm{HPT}}, \Delta\Gamma_{\mathrm{HPT}}, \Delta\eta_{\mathrm{LPT}}, \Delta\Gamma_{\mathrm{LPT}}]^{\mathsf{T}}, \tag{2.2}$$

expressed in percent relative to a new engine. $\mathbf{x}$ is what maintenance actually wants to know, and it is not directly measurable on the wing.

2.4.2 The linear model

What *is* measurable are deviations $\Delta\mathbf{z}$ of the corrected parameters from a *baseline* (the value a new engine would show at the same operating condition $\mathbf{u}$: altitude, Mach number, N1 setting, ambient temperature deviation, bleed status). For the small deviations typical of in-service deterioration, measurement and health deviations are related linearly:

$$\Delta\mathbf{z} = \mathbf{H}(\mathbf{u})\,\mathbf{x} + \mathbf{S}\,\mathbf{b} + \mathbf{v}, \qquad \mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{R}), \tag{2.3}$$

where $\mathbf{H}$ is the *influence coefficient matrix* (ICM) —the Jacobian $\partial\mathbf{z}/\partial\mathbf{x}$, i.e. a table of "if the HPT loses 1 % efficiency, EGT rises by this much and fuel flow by that much"— computed by finite differences on the thermodynamic model at the operating condition $\mathbf{u}$; $\mathbf{b}$ collects sensor biases and $\mathbf{S}$ is the *selection matrix* that routes each bias to its own measurement row — one column per biased sensor, all zeros except a single one, so that a bias on the EGT probe shifts only the EGT equation. Estimating $\mathbf{b}$ alongside $\mathbf{x}$ is optional and costs observability; an undetected drift therefore fools the estimator, which is exactly the sensor fault Chapter 5 injects. Finally $\mathbf{v}$ is measurement noise with covariance $\mathbf{R}$.

2.4.3 Estimation, and why it is hard

Inverting Eq. (2.3) is an ill-posed problem. With 4 cockpit measurements and 10 unknowns the system is underdetermined ($\text{rank}(\mathbf{H}) \leq 4$): infinitely many health states explain the same measurements. The weighted-least-squares (WLS) answer regularizes the inversion with prior knowledge of plausible deterioration:

$$\hat{\mathbf{x}} = \left(\mathbf{H}^{\mathsf{T}}\mathbf{R}^{-1}\mathbf{H} + \lambda\,\mathbf{P}_0^{-1}\right)^{-1}\mathbf{H}^{\mathsf{T}}\mathbf{R}^{-1}\Delta\mathbf{z}, \tag{2.4}$$

where the two regularization pieces play distinct roles: $\mathbf{P}_0$ (a diagonal matrix of expected deterioration magnitudes squared) sets the *shape* of the prior — which parameters are allowed to move more — while the scalar λ sets its *strength*, trading fidelity to the measurements against fidelity to the prior. λ is a tuning knob of the traditional pipeline and receives the same optimization budget as the AI's hyperparameters (Section 3.4).

Because deterioration evolves slowly over thousands of flights, tracking it as a *random walk* with a Kalman filter improves on flight-by-flight estimation. The Kalman filter is a recursive estimator that blends the previous health estimate with each new measurement, weighting each by its uncertainty:

$$\mathbf{x}_k = \mathbf{x}_{k-1} + \mathbf{w}_k, \quad \mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}); \qquad \mathbf{z}_k = \mathbf{H}(\mathbf{u}_k)\,\mathbf{x}_k + \mathbf{v}_k, \tag{2.5}$$

with process noise $\mathbf{Q}$ calibrated to realistic degradation rates and the state optionally augmented with sensor biases [2]. At recorded compressor-wash events the recoverable part of the state is reset.

2.4.4 Smearing: a two-parameter toy example

Before the formal statement, a miniature GPA problem shows the failure mode this whole project revolves around. Suppose only two health parameters (HPC and HPT efficiency) and two measurements (EGT and W_f deviations, in %), with influence matrix

$$\mathbf{H} = \begin{pmatrix} 1.0 & 0.9 \\ 0.5 & 0.5 \end{pmatrix},$$

whose columns —the two fault signatures— point in nearly the same direction. Let the truth be an HPC fault only: $\mathbf{x} = (-1, 0)^{\mathsf{T}}$, giving exact measurements $\Delta\mathbf{z} = (-1.0, -0.5)^{\mathsf{T}}$, and indeed $\mathbf{H}^{-1}\Delta\mathbf{z} = (-1, 0)^{\mathsf{T}}$ recovers it. Now perturb the first measurement by just $+0.1$ (a 10 % error, comparable to one noise standard deviation):

$$\mathbf{H}^{-1}\begin{pmatrix} -0.9 \\ -0.5 \end{pmatrix} = \begin{pmatrix} 10 & -18 \\ -10 & 20 \end{pmatrix}\begin{pmatrix} -0.9 \\ -0.5 \end{pmatrix} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}.$$

The estimate now says: HPC healthy, *HPT* degraded by 1 % — a single noise-sized error flipped the diagnosis to the wrong component entirely. Nothing is wrong with the algebra; the information simply is not in the measurements when signatures nearly coincide.

2.4.5 Smearing in general

That is *smearing*: when $\mathbf{H}$ is ill-conditioned —fault signatures nearly parallel— noise makes the estimator distribute a fault that is physically concentrated in one component across several, or attribute it to the wrong one; it is the classical weakness of linear GPA [9]. A singular-value decomposition (SVD) of the ICM quantifies this weakness (rank, condition number, angles between fault signatures) and defines the "confusable" subset used by hypothesis H2: fault pairs whose signatures are less than 15° apart.

Table 2.2: Degradation mechanisms modeled in this project and their typical effects, per the literature [10, 11].

Mechanism	Component	Typical effect	Time profile	Recoverable
Fouling (dirt buildup)	fan, LPC, HPC	$\Delta\Gamma$ $-1\ldots-4\,\%$; $\Delta\eta$ $-0.5\ldots-2.5\,\%$	saturating exponential	wash recovers 30–70 %
Erosion	compressors	$\Delta\eta$ $-0.5\ldots-1.5\,\%$	slow linear	no
Tip-clearance opening	HPC/HPT	$\Delta\eta$ $-0.5\ldots-1.5\,\%$	fast initially, then slow	no
Hot-section deterioration	HPT	$\Delta\eta$ $-1\ldots-3\,\%$; $\Delta\Gamma$ $+1\ldots+2\,\%$	linear, accelerating	no
Foreign-object damage (FOD)	fan/HPC	step $\Delta\eta$ $-0.5\ldots-2\,\%$	step (Poisson arrivals)	no
Sensor drift	any probe	e.g. EGT $+1\ldots+5$ °C per 1000 cycles	ramp	recalibration

2.5 The physics of degradation

The aggregate observable effect is the erosion of the *EGT margin*: the distance between the EGT reached on a hot-day takeoff and its certified limit. A new engine has 70–100 °C of margin; when the margin reaches zero the engine can no longer legally produce rated takeoff thrust on a hot day and must come off the wing. Its characteristic pattern over an engine's life —fast early loss, slower steady erosion, sawtooth recoveries at each compressor wash— is what the fleet generator of phase F2 must reproduce. Table 2.2 summarizes the mechanisms, their signatures and their time profiles; its magnitudes parameterize that generator.

Chapter 3

Methodology and testbed architecture

This chapter in brief: the project is six chained phases, each ending in a pass/fail gate (Fig. 3.1). Four safeguards keep the AI-versus-traditional comparison honest; one section explains every machine-learning tool by name before it is used; and the software inventory maps each program to its role. Read Section 3.3 even if you skip the rest.

3.1 Overall architecture

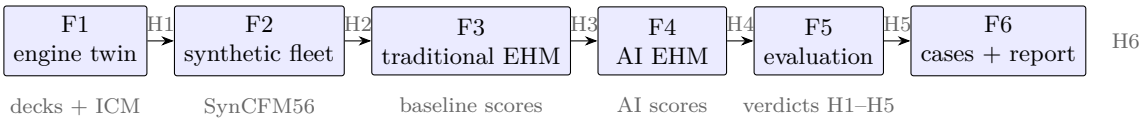

Figure 3.1: The six project phases, their gates (H1–H6) and their key artifacts. Everything to the left of an arrow must pass its gate before the next phase starts.

The testbed is organized in six chained phases, each closed by a milestone (*gate*) with a measurable acceptance criterion:

F1. Thermodynamic twin of the CFM56-7B26 in pyCycle: design point, off-design behavior, baseline decks over the flight envelope (pre-computed tables of what a healthy engine reads at every operating condition) and the influence coefficient matrix. Gate H1: error $\leq 3\,\%$ against certified data at the anchor points.

F2. Synthetic fleet generator (SynCFM56): per-mechanism health trajectories, a hierarchical fleet of $\sim$100 engines run to failure, a sensor model (noise, bias, quantization, missing data) and ACARS-style snapshots with ground truth. Gate H2: dataset audited for realism and difficulty, no leakage between data splits, and a written datasheet.

F3. Traditional EHM reference: baselines and smoothed deviations, trend monitoring with persistence rules, WLS GPA (Eq. (2.4)), Kalman GPA (Eq. (2.5)), expert-rule fault isolation, and RUL by robust extrapolation of the EGT margin. Gate H3: full metrics on the test set.

F4. AI-based EHM: anomaly detection (autoencoders, Isolation Forest), multi-class fault diagnosis (gradient boosting, neural networks), RUL prognosis (LSTM/TCN/Transformer sequence models), the physics-informed hybrid, conformal uncertainty and physical explainability (PCS). Gate H4.

F5. Pre-registered comparative evaluation: common metrics, statistical tests, ablations, and sim-to-real validation on N-CMAPSS. Gate H5.

F6. Case studies, interactive dashboard, and final writing. Gate H6.

3.2 Machine learning without prior background

Nothing in this document assumes machine-learning training. Six ideas carry everything:

Learning from data. A machine-learning model is a mathematical function with many adjustable numbers inside (its *parameters*). "Training" means showing it examples (here: deviation histories with known outcomes, because the fleet is synthetic) and letting an algorithm nudge those numbers until the function's outputs match the known answers. Nobody programs the rules; the rules emerge from the examples.

Train / validation / test. Three disjoint groups of engines with three jobs. *Training* engines are the textbook the model studies. *Validation* engines are practice exams: used to choose between model variants and settings, never to adjust parameters directly. *Test* engines are the final exam, sat exactly once — any peeking turns the grade into fiction. The same discipline is applied to the traditional pipeline's thresholds.

Overfitting. A model with enough capacity can memorize its textbook — perfect on training engines, useless on new ones. That is why every number that matters in this document comes from engines the models never saw, and why splitting by *engine* (not by flight) is non-negotiable: flights of the same engine are near-duplicates.

Neural networks, in one paragraph. A neural network is a chain of simple layers: each layer multiplies its inputs by adjustable weights, sums them, and passes the result through a mild nonlinearity. Stacked layers can approximate very general input–output relationships. Training adjusts the weights by *gradient descent*: compute how wrong the output is, compute in which direction each weight should move to reduce that error, take a small step, repeat over the dataset for a number of *epochs* (passes). A *recurrent* network (like the GRU used here) additionally carries a small memory from one time step to the next, so it can read a sequence — an engine's deviation history — the way a reader accumulates context along a sentence.

Hyperparameters and tuning. Settings the training process itself does not adjust — network size, learning-rate, window lengths, alert thresholds — are *hyperparameters*. Choosing them by trial and error on validation data is *tuning*; automating that search is what Optuna does. Both EHM families received exactly 50 tuning trials (Section 3.4): the comparison is between tuned practitioners, not between a tuned one and a default one.

Seeds and variance. Training involves randomness (initial weights, sampling order). A *seed* fixes that randomness so runs are repeatable; training with several seeds and reporting the spread separates a real effect from a lucky draw — which is exactly how the hybrid experiment's negative result was established.

3.3 The machine-learning toolbox, briefly

With those six ideas in hand, the specific techniques, one paragraph each:

Autoencoder (anomaly detection). A neural network trained to compress and then reconstruct *healthy* data. Fed a degraded snapshot, it reconstructs it poorly; the reconstruction error is the anomaly score. No fault labels needed.

Isolation Forest (anomaly detection). An ensemble of random trees; anomalous points are isolated in fewer random splits than normal ones. A classical non-neural baseline.

Gradient boosting / XGBoost (diagnosis). An ensemble of small decision trees, each correcting the errors of the previous ones. State of the art on tabular data; predicts the fault class of a snapshot window.

LSTM, TCN, Transformer (RUL prognosis). Three sequence-model families —recurrent, convolutional and attention-based— that consume a sliding window of past snapshots and regress the remaining useful life. The v0 suite implements a GRU (recurrent); the cross-family comparison belongs to the F5 tuned round.

Conformal prediction (uncertainty). A distribution-free wrapper: from the model's errors on a calibration set it builds prediction intervals with a mathematically guaranteed coverage rate (e.g. 90 %), regardless of the underlying model.

SHAP (explainability). Assigns each input feature its contribution to one specific prediction (a game-theoretic attribution). The PCS (contribution C5) projects these attributions through the ICM into health-parameter space to test their physical coherence.

Optuna (tuning). An automated hyperparameter search library; a "trial" is one training run with a candidate configuration. Both EHM families get the same trial budget.

Supporting infrastructure named later: *DVC* versions datasets the way git versions code; *MLflow* records every training run (configuration, metrics, artifacts) so experiments are traceable and repeatable.

3.4 Methodological safeguards

The credibility of an AI-versus-traditional comparison depends on eliminating the biases that plague the literature. Four safeguards are adopted:

Pre-registration. Hypotheses (Table 1.1), metrics, data splits (with file hashes), statistical tests and the stopping rule are frozen in the repository under a git tag (`prereg-v1`, document `docs/prereg-v1.md`) *before* the confirmatory evaluation. This borrows a standard practice from medicine: you state what would convince you before looking at the answer, so you cannot move the goalposts afterwards. The frozen document also discloses every exploratory test-set exposure that preceded it.

Symmetric tuning budget. Every method has knobs. The traditional pipeline's (alert thresholds, regularization λ, Kalman $\mathbf{Q}$, smoothing windows) and each AI family's hyperparameters are tuned with the *same number* of automated search trials (Optuna), using only the training and validation engines.

Split by engine. The fleet is divided 70/10/20 engines into training, validation and test sets, and no engine ever appears in two sets. Splitting by flight instead of by engine —a common mistake— would let a model memorize an engine's individual quirks and score unrealistically well. An automated leakage check runs in continuous integration.

A strong baseline. The traditional pipeline is implemented with the same care as the AI, every expert rule documented with its physical justification; smearing is *measured* against ground truth, not assumed.

3.5 Reproducibility and infrastructure

The entire project is a public repository[1] with a reproducible environment: Python 3.11 managed with `uv` and a fully pinned dependency lockfile; pyCycle 4.4 on OpenMDAO [12]; declarative YAML configuration; and automated tests (pytest) executed by continuous integration —a service that re-runs the whole test suite on every change, so a regression cannot land silently. Data and experiments of later phases will be versioned with DVC and MLflow respectively.

Three concrete decisions illustrate the intended standard of rigor:

- **A versioned calibration contract.** The model's numerical targets (Table 2.1) live in a single file (`conf/cfm56_7b_targets.yaml`) carrying value, tolerance, source and status (`VERIFIED`/`TO_VERIFY`) per entry. Both the model runners and the tests read that file: changing a target is a visible version-control event, never a silent edit.
- **Dependency canary tests.** During environment setup, an incompatibility between pyCycle 4.4 and numpy ≥ 2 was discovered (a failure inside the tabular thermodynamics package). The fix pins `numpy<2`, and a smoke test that runs a complete engine cycle executes on every CI run, so any future dependency upgrade that breaks the solver fails loudly and immediately.
- **Auto-generated evidence.** Every result figure and table in this document is produced by a script (`scripts/make_report_assets.py`) that solves the model and writes the PDFs, the LaTeX table files (`paper/report/tables/*.tex`) and a provenance file (`assets.json`). No result number is ever copied by hand.
- **Compute-efficiency norms.** The repository carries binding engineering norms (`docs/engineering-norms.md`): long decomposable computations run in parallel across CPU cores (thermodynamic sweeps and ICM columns use one worker process per core — these are scalar Newton solves that gain nothing from a GPU), while the tensor workloads of phase F4 (network training, SHAP) run on the GPU (Apple-silicon MPS backend on the development machine).

3.6 Software architecture and computational pipeline

This section is the complete inventory of the project's code: what each program does, how it does it, and what flows between them. It is updated whenever code lands.

3.6.1 Library modules (`src/ehmbrain/`)

`perf/cycle.py` — the engine model. Defines four model classes and their builders.

- `CFM56(pyc.Cycle)`: one thermodynamic point. Builds the component graph (inlet, fan, splitter, booster, HPC with three bleed ports, post-compressor bleed, combustor, cooled HPT/LPT, ducts, two convergent nozzles, two shafts, performance block) and its balance system. In *design* mode the four balances solve mass flow for thrust, fuel–air ratio for T_{t4}, and both turbine expansion ratios for zero net shaft power. In *off-design* mode they solve mass flow and bypass ratio to hold the design nozzle throat areas, both spool speeds for shaft power balance, and fuel–air ratio according to the throttle mode: `T4` (rating temperature), `percent_thrust` (a fraction of a reference thrust) or `N1` (hold fan speed, the CFM56 rating parameter, implemented through a passthrough component because the speed is itself another balance's output). Each point carries its own Newton solver (tolerance 10^{-8}, Armijo–Goldstein line search, subsystem solves enabled).

[1] `https://github.com/cedarcreeks/EHMbrAIn`

- `MPCFM56(pyc.MPCycle)`: DESIGN + the four SLS anchors A2–A5 as thrust-matched `percent_thrust` points. Used for calibration (Section 4.4).
- `MPCFM56Study`: DESIGN + one off-design point whose five turbomachine map scalars (efficiency and flow) pass through multiplier components (`health_<comp>.eta_fac/flow_fac`); setting a factor to $1 + \Delta$ imposes a health deviation. Used for the ICM and, in F2, for degraded-fleet snapshots.
- `MPCFM56Deck`: DESIGN + `OD_max` (maximum thrust at the rating T_4) + `OD_pt` (a commanded fraction of that maximum). Used for the baseline decks.
- Builders (`build_problem`, `build_study_problem`, `build_deck_problem`) return solver-ready problems with all design inputs and balance initial guesses set; `solve_anchors()` encapsulates the idle power continuation ($0.30 \to 0.07$); `snapshot()` reads the full sensor set at a point; `set_health()` imposes a health-deviation dictionary.

`perf/icm.py` — **GPA sensitivity analysis.** `compute_icm()` builds a Study problem per health parameter (in parallel worker processes, norm N1), perturbs it $\pm 0.5\,\%$ at constant N1 and forms the central-difference column in %-per-%; `svd_report()` returns rank, singular values and condition number; `signature_angles()` the pairwise angles between fault signatures; and `confusable_pairs()` the sub-15° subset that instantiates hypothesis H2.

`common/corrections.py` — **corrected parameters.** ISA atmosphere (`isa_static`), ram-corrected θ/δ at the fan face (`theta_delta`), the exponent regression `fit_correction_exponents()` ($\log Y$ against a cubic polynomial in log corrected speed plus $a \log\theta + b \log\delta$, solved by least squares) and `correct()` to apply the fitted law.

3.6.2 Executable scripts (`scripts/`)

Each script is a pipeline stage writing versioned artifacts under `data/processed/`:

Script	Function and outputs
`run_design_point.py`	Solves the design point, checks the five WP1.1 acceptance windows against the contract; nonzero exit on failure.
`run_anchors.py`	Solves DESIGN + A2–A5 with the idle continuation and compares fuel-flow/OPR predictions to the EEDB (gated tolerances); prints the verdict table.
`make_decks.py`	Envelope sweep for the baseline decks: independent (altitude, Mach) blocks in parallel processes, warm continuation inside a block, cold-rebuild recovery (norm N2), then a per-channel parallel leave-one-out audit. → `baseline_deck.parquet`, `deck_report.json`.
`make_corrected_baseline.py`	Fits the correction exponents on the deck and re-runs the leave-one-out audit in corrected space. → `corrected_baseline.json`.
`make_icm.py`	ICM at the six-point operating grid (columns in parallel), SVD and confusability analysis. → `icm_<point>.npz`, `icm_report.json`.
`cea_crosscheck.py`	Re-solves the design point with CEA thermodynamics and reports deltas against tabular. → `cea_crosscheck.json`.
`make_report_assets.py`	Regenerates every figure and table of this document from the model and the artifacts above (norm N4). → `paper/report/figures/*.pdf`, `paper/report/tables/*.tex`, `assets.json`.

Later phases added, under the same contract-artifact-evidence pattern:

`datagen/ trajectories.py` (per-mechanism health profiles, exact wash sawtooth, multi-episode acute faults), `fleet.py` (engine sampling, EGT-margin end of life, leakage-free splits),

`sensors.py` (noise/quantization/drift/dropout), `snapshots.py` (ACARS-style two-report generator via the F1 linearization). Driver: `make_fleet.py`; audits: `audit_nonlinearity.py`, `audit_dataset.py`.

`trad/ pipeline.py`: baseline deviations, Holt smoothing with innovations, CUSUM and dual-EWMA gap detectors, WLS and wash-aware Kalman GPA, nearest-signature isolation, Theil–Sen RUL. Driver: `run_trad.py`.

`ai/ data.py` (shared feature builder — same deviations as `trad/`), `models.py` (autoencoder, GRU RUL net, MPS-aware training loops). Drivers: `run_ai.py` (detection/diagnosis/RUL + conformal), `run_hybrid.py` (stacking ablation), `run_pcs.py` (Physics-Consistency Score), `tune_detection.py` (validation sweep).

The data flow is linear: `conf/*.yaml` (the contracts) feed the model and generator; runners and `make_*` scripts write `data/processed/*` artifacts; `make_report_assets.py` turns artifacts into report evidence; the tests gate every stage.

3.6.3 Test suite (`tests/`)

Thirty-six tests, all run in CI on every change: environment smoke (the vendored pyCycle example end-to-end); design-point convergence and acceptance windows; anchor convergence, fuel-flow and OPR tolerances, and N1/N2 redlines; the Study self-consistency property; ICM physical signs and rank-3 cockpit underdetermination; the corrections module (ISA, θ/δ, exponent-fit recovery on synthetic truth); trajectory physics (wash sawtooth ratios and regrowth, FOD steps, reproducibility, split integrity); the sensor and snapshot models (noise statistics, quantization, drift ramps, dropout fraction, end-to-end consistency); traditional-pipeline primitives (Holt trend and innovations, CUSUM step-not-trend, gap-detector ramp capture, WLS recovery, Kalman observation-space convergence, Theil–Sen); and AI primitives (feature fill, window alignment, device selection, autoencoder separation, GRU countdown learning).

3.7 Replication guide

This document and the repository are designed to be used together: every result chapter names the script that produced its evidence, and this section gives the complete replication path. **Reference machine for every wall-clock figure in this document**: an Apple M5 desktop chip (10 cores, 16 GB unified memory), with tensor training on its GPU through the torch MPS backend — consumer hardware, deliberately. Total cost: about **11 minutes** end to end (Table 3.1; per-stage breakdown measured by `scripts/benchmark_pipeline.py`, norm N5).

Setup (once).

1. Clone `github.com/cedarcreeks/EHMbrAIn` and install `uv` (the Python package manager).
2. `uvsync` — reproduces the exact pinned environment (Python 3.11, pyCycle 4.4, torch with MPS).
3. `uvrunpytest` — 36 tests must pass before anything else; they gate every claim in this document.

Pipeline (in order). `makeall` runs everything below and rebuilds this document (gate H6); the stage-by-stage mapping to report evidence is:

Command (`uvrunpython\ldots`)	Produces	Feeds
`scripts/run_design_point.py`	design-point check	Tables 4.1 and 4.2
`scripts/run_anchors.py`	calibration verdicts	Table 4.3 and Fig. 4.2
`scripts/make_decks.py`	baseline decks + LOO	Table 4.5
`scripts/make_corrected_baseline.py`	corrected-space exponents	Table 4.4
`scripts/make_icm.py`	ICM grid + observability	Fig. 4.4 and Tables 4.6 and 4.7
`scripts/cea_crosscheck.py`	thermo cross-check	Table 4.8
`scripts/make_fleet.py`	SynCFM56 fleet	Table 5.1 and Figs. 5.1 and 5.2
`scripts/audit_dataset.py`	difficulty + realism audits	Table 5.3
`scripts/audit_nonlinearity.py`	linearization audit	Table 5.2
`scripts/run_trad.py`	traditional metrics	Table 6.1 and Figs. 6.1 and 6.2
`scripts/run_ai.py`	AI metrics + conformal	Table 7.1 and Fig. 7.1
`scripts/run_hybrid.py`	hybrid ablation	Fig. 7.2
`scripts/run_pcs.py`	PCS attribution study	Section 7.5
`scripts/tune_f5.py` + `scripts/f5_confirm.py`	tuned configs; H1–H5 verdicts	Table 8.1
`scripts/sim_to_real.py`	external-benchmark ranking check	Section 8.7
`scripts/benchmark_pipeline.py`	compute times (norm N5)	Table 3.1
`scripts/make_case_studies.py`	five case-study figures + facts	Chapter 9
`scripts/make_report_assets.py`	*every* figure and table above	this document

Two platform notes a replicator needs (both learned the hard way, both in the decision register): torch-MPS training runs must execute in the *foreground* on macOS (backgrounded runs segfault), and XGBoost cannot coexist with torch-MPS in one process (scikit-learn's histogram gradient boosting is used instead).

3.7.1 Compute cost on the reference machine

Table 3.1: Wall-clock time per stage on the reference desktop machine (Apple M5, 10 cores, 16 GB, torch MPS backend). Cold-start subprocess timings — what a replicator actually pays. Auto-generated per norm N5.

Stage	Script	Device	Wall time [s]
design point	`scripts/run_design_point.py`	CPU	2
sls anchors	`scripts/run_anchors.py`	CPU	13
baseline decks	`scripts/make_decks.py`	CPU	214
corrected baseline	`scripts/make_corrected_baseline.py`	CPU	1
icm grid	`scripts/make_icm.py`	CPU	71
fleet generation	`scripts/make_fleet.py`	CPU	40
audit dataset	`scripts/audit_dataset.py`	CPU	12
audit nonlinearity	`scripts/audit_nonlinearity.py`	CPU	20
traditional pipeline	`scripts/run_trad.py`	CPU	106
ai suite	`scripts/run_ai.py`	CPU+GPU	27
hybrid ablation	`scripts/run_hybrid.py`	CPU+GPU	104
pcs	`scripts/run_pcs.py`	CPU+GPU	46
Full pipeline			**657**

The entire evidence base of this document regenerates in 657 seconds — about eleven minutes — on the Apple M5 reference machine, GPU stages included. This is the quantitative form of transferability rule 4: if it only ran on a cluster, it would not be adoptable.

3.8 Comparison metrics

Both families are scored with identical metrics. For detection: lead time (how many flights before the ground-truth event the method raises a sustained alert), false alarms per thousand flights, F1 and precision-recall AUC. For isolation: top-1/top-2 accuracy, macro-F1, a *smearing index* (the fraction of $\|\hat{\mathbf{x}}\|_1$ assigned to healthy components) and the PCS. For health estimation: RMSE of $\hat{\mathbf{x}}$ against ground truth. For RUL: RMSE, MAE, the NASA PHM08 score

$$S = \sum_i s_i, \qquad s_i = \begin{cases} e^{-d_i/13} - 1 & d_i < 0 \\ e^{d_i/10} - 1 & d_i \geq 0 \end{cases}, \qquad d_i = \widehat{\mathrm{RUL}}_i - \mathrm{RUL}_i, \tag{3.1}$$

which penalizes late predictions more than early ones (a late removal risks an in-flight shutdown; an early one merely wastes some life). Two further prognostic metrics need definitions: α–λ *accuracy* is the fraction of predictions falling within $\pm\alpha$ (here 20 %) of the true RUL when evaluated at specified life fractions λ (here 50, 70 and 90 % of the way to failure) — it asks "was the prediction acceptable at mid-life? near the end?"; the *prognostic horizon* is the earliest moment from which the prediction stays within the $\pm\alpha$ band for the rest of the engine's life — longer horizons mean earlier trustworthy warnings. For uncertainty: empirical versus nominal interval coverage (does the "90 %" interval actually contain the truth 90 % of the time?) and mean interval width (narrow honest intervals are the goal; wide intervals are trivially calibrated but useless).

Chapter 4

Thermodynamic twin of the CFM56-7B26

This chapter in brief: a virtual CFM56-7B26 is built in pyCycle by adapting a validated NASA example, calibrated against certified data (its fuel-flow predictions land within 3 % of measured test-bed values), and then interrogated for the two artifacts everything else consumes: the healthy-engine baseline decks and the influence coefficient matrix, whose geometry quantifies exactly how blind the cockpit sensor set is.

This chapter documents the completed part of phase F1: the CFM56-7B26 performance model in pyCycle [5], its design point (work package WP1.1) and its off-design calibration against certified measured data (WP1.2). All result figures and tables are produced by the evidence-generation script (Section 3.5).

4.1 What a cycle model is

A 0-D *cycle model* represents the engine as a chain of thermodynamic components —inlet, fan, compressors, combustor, turbines, nozzles— each transforming the air's pressure, temperature and composition according to its governing equations, without resolving any internal geometry. Each turbomachine carries a *component map*: a table, obtained from rig tests or scaled from a similar machine, giving its efficiency and flow as a function of rotational speed and operating point. The model runs in two modes:

Design mode sizes the engine: given targets (thrust, component pressure ratios, turbine inlet temperature), it computes the flow areas, map scale factors and shaft-work balance that a machine meeting those targets must have.

Off-design mode answers the service question: given the now-fixed geometry, what does the engine do at any other flight condition and thrust setting? This is the mode the EHM baseline decks and the influence coefficient matrix are built from.

pyCycle solves both modes with a Newton solver: it iterates on a set of unknowns (mass flow, fuel–air ratio, spool speeds, map operating points) until a set of residual equations (nozzle-area match, shaft power balance, thrust target) is driven to zero. Newton solvers are powerful but fragile far from a good initial guess —a fragility that shaped several decisions below.

4.2 Strategy: adapt, don't build from scratch

pyCycle ships a validated example of a two-spool, separate-flow, cooled high-bypass turbofan (`high_bypass_turbofan.py`) with the full component graph, the turbine-cooling bleed network,

shaft balances and off-design solver wiring already working. Building the CFM56 from a blank file would re-derive all of that plumbing and typically dies in Newton divergence. Instead, a *morphing* protocol is adopted: the example is vendored into the repository as a known-good regression reference, and the model is steered toward the CFM56-7B26 by changing **at most three parameters per step**, requiring convergence at every step before the next. Every converged step is a commit; if a step diverges, the change is bisected.

All design decisions were fixed in writing *before* coding (`docs/f1-model-spec.md`): the -7B26 variant; tabular thermodynamics for speed with a CEA cross-check planned (CEA is NASA's chemical-equilibrium code —more accurate, roughly an order of magnitude slower); the design point at cruise, Mach 0.78 / 35 000 ft / ISA (the "aerodynamic design point" convention: an engine is designed for cruise, where it spends most of its life, and takeoff is an off-design condition); pyCycle's generic component maps, scaled —no public CFM56 maps exist, a declared limitation; thrust set through corrected N1 in off-design (the -7B is an N1-rated engine); pyCycle-native units inside the model with SI only at the boundary; and a table of anticipated solver failure modes, each with a planned countermeasure —one of which proved prophetic (Section 4.4.3).

4.3 Design point (WP1.1)

The cycle graph comprises: flight conditions, inlet, fan, flow splitter, booster, HPC with three bleed ports (HPT vane and blade cooling, customer bleed), a post-compressor bleed feeding the LPT, combustor, cooled HPT and LPT, ducts with pressure losses, convergent core and bypass nozzles, the two shafts, and an accessory power extraction of 250 hp on the HP shaft (driving the aircraft's generators and pumps). In design mode, four implicit balances close the cycle: the inlet mass flow W meets the thrust target; the fuel–air ratio (FAR) meets the target turbine inlet temperature T_{t4}; and the HPT and LPT expansion ratios zero the net power on each shaft (each turbine produces exactly what its compressors and losses consume).

Table 4.1: Design-point results (Mach 0.78, 35 000 ft, ISA). Auto-generated from the model.

Quantity	Value at the design point
Net thrust F_n	24.38 kN (5480 lbf)
TSFC	0.6214 lb/(lbf·h)
Bypass ratio (BPR)	5.10
Overall pressure ratio (OPR)	30.03
T_{t4}	1587 K (2857 °R)
Inlet mass flow W	142.0 kg s^{-1}
Fuel–air ratio (FAR)	0.0251
HPT expansion ratio	3.586
LPT expansion ratio	4.288
Total cooling/bleed fraction	0.239

Table 4.2: Total conditions at each gas-path station at the design point. Auto-generated.

Station	Location	P_t [bar]	T_t [K]
2	Fan inlet	0.358	245.5
21	Fan exit	0.603	289.5
25	HPC inlet	1.149	354.2
3	HPC exit	10.739	703.9
4	Burner exit	10.159	1587.2
45	HPT exit	2.833	1141.6
5	LPT exit	0.657	806.3

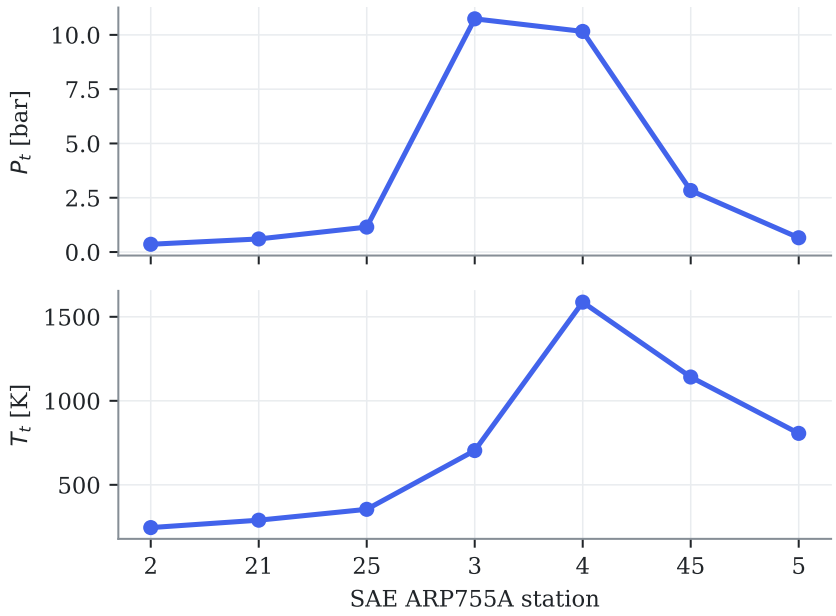

Figure 4.1: Total pressure and temperature along the gas path at the design point. Compression peaks at station 3 (∼11 bar, Table 4.2), heat addition at station 4, and work extraction across stations 45 and 5.

The design point converges with a Newton residual below 10^{-6} and satisfies all five acceptance windows defined a priori: BPR 5.10 ± 0.2 (the EEDB target), OPR within its window, T_{t4} within a plausible band, total cooling-plus-bleed fraction between 18 and 25 % of core flow, and thrust exactly on target (Table 4.1). The resulting TSFC (≈ 0.62 lb/(lbf·h)) lands within 2 % of the publicly quoted cruise value for this engine *without having been used as a calibration target* —an encouraging consistency check.

4.4 Calibration against measured data (WP1.2)

4.4.1 Data sources and philosophy

Off-design calibration uses exclusively **measured, certified data** (Table 2.1). The TCDS provides limits and rated thrusts. The ICAO Engine Emissions Databank provides something less well known: sea-level test-bed *measured fuel flows* at the four landing-takeoff-cycle thrust settings (100, 85, 30 and 7 % of rated thrust F_{00}), plus the engine pressure ratio at rated output (27.61) —all for the exact -7B26 variant, traceable to certification testing, and free. Most academic engine models ignore this source.

The anchor points are solved *thrust-matched*: the fuel–air ratio balance is set to hold $F_n = \mathrm{PC} \cdot F_{00}$ (PC being the power fraction). The model's fuel flow at each anchor is therefore a genuine **prediction** confronted with the EEDB measurement —not a quantity fitted to it.

4.4.2 Calibration iterations

The first run (model freshly adapted from the example, not yet calibrated to the SLS data) already predicted takeoff fuel flow within −2.1 %, but showed two quantitative defects:

1. **N2 above its redline at rated thrust**: 15 311 rpm against the certified limit of 15 183 rpm. Cause: the reference speed used to scale the generic HPC map —the map's speed-versus-flow shape is not the real compressor's, so the speed labels it implies must be anchored

deliberately. Fix: HP-shaft design reference from 14 324 to 13 940 rpm, placing rated-thrust N2 at 14 915 rpm (≈ 103 %), inside the limit.

2. **Takeoff pressure ratio 28.99 versus the measured 27.61** (+5.0 %). Fix: HPC design pressure ratio from 9.81 to 9.35. The measured value takes precedence over the initial derived estimate, and the cruise OPR becomes a consequence (≈ 30).

Table 4.3: SLS anchors versus the ICAO databank measurements after calibration. W_f model is the model's prediction at imposed thrust. Auto-generated.

Anchor	$\%F_{00}$	F_n [kN]	W_f model	W_f EEDB	error [%]	tol. [%]	gated
A2 Takeoff	100	117.0	1.204	1.221	-1.40	3.0	✓
A3 Climbout	85	99.4	0.977	0.999	-2.23	3.0	✓
A4 Approach	30	35.1	0.328	0.338	-3.08	5.0	✓
A5 Idle	7	8.2	0.125	0.113	+10.57	5.0	—

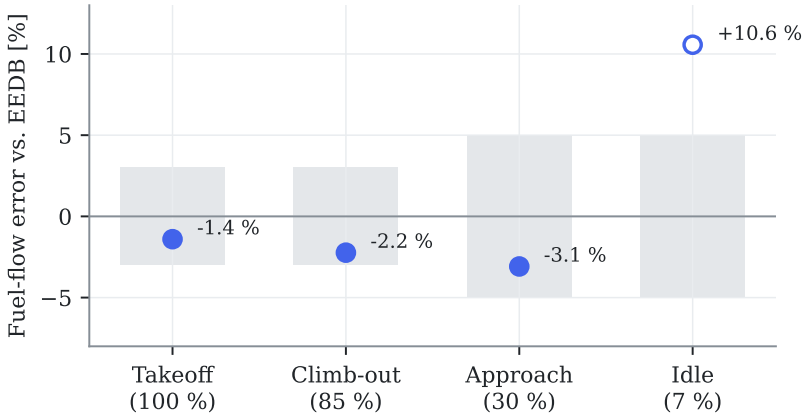

Figure 4.2: Fuel-flow prediction error at each anchor against the EEDB measurement. Gray bands are the contract tolerances (±3 % at takeoff and climb-out, ±5 % at approach and idle); the hollow marker (idle) converges but sits outside tolerance and is reported without blocking the milestone (Section 4.4.3).

After both corrections (Table 4.3 and Figs. 4.2 and 4.3), the three gated anchors meet their tolerances: takeoff at −1.40 % fuel-flow error with the pressure ratio within +0.3 % of the measured value and both shaft speeds inside their redlines; climb-out at −2.23 %; approach at −3.08 %.

4.4.3 Idle convergence: diagnosis and cure

The idle point (7 % of F_{00}, static) systematically diverges from a cold start. At such low power the nozzle pressure ratios approach unity —the exhaust barely pushes— and the nozzles' internal static-pressure sub-solver bounces against its lower bound, leaving the global Newton iteration without a descent direction. Worse, the failed attempt leaves the point's internal states (flow stations, map positions) corrupted, so re-seeding only the balance variables does not rescue it: the idle point must *never* start cold at its final setting. The cure —anticipated as a generic countermeasure in the specification— is **power continuation**: the point starts at 30 % of F_{00} (where it converges from cold, just like the approach anchor) and walks down warm through $0.30 \to 0.20 \to 0.14 \to 0.10 \to 0.08 \to 0.07$, re-solving at each step. The logic is encapsulated in the library (`solve_anchors()`) and covered by the tests.

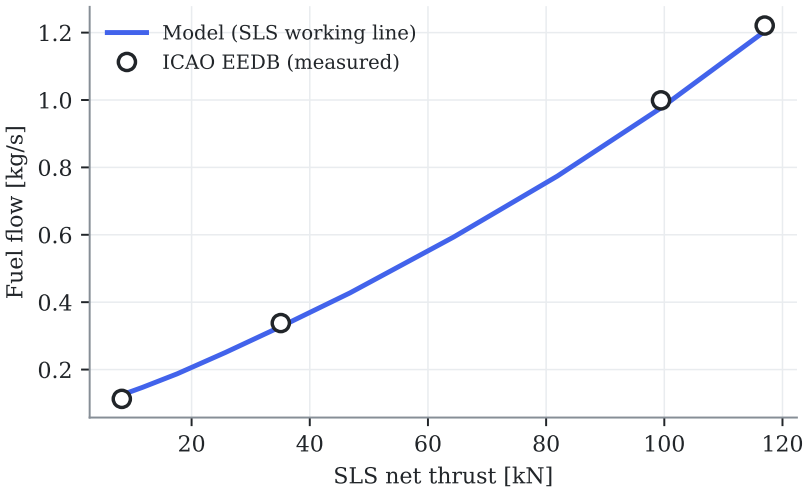

Figure 4.3: The model's sea-level-static working line (a continuous sweep of thrust setting) against the four measured ICAO databank points. The curvature at low power reflects the degraded accuracy of generic maps far from their scaling point.

With continuation, idle converges and predicts fuel flow with +10.6 % error. This is deep map-extrapolation territory —the generic map's speed-flow shape at very low power is not the real compressor's— a known limitation of the approach; the anchor is **reported but not gated** for milestone H1, a decision recorded in the calibration contract (`gated:~false`).

4.5 Baseline decks (WP1.3)

Both EHM pipelines need a *baseline*: what a healthy engine reads at any operating condition, so that measured values can be turned into deviations. The baseline is produced by sweeping the healthy model over a realistic flight envelope —altitude blocks with their plausible Mach ranges (no Mach 0.8 at sea level), ambient temperature offsets of 0, +15 and +30 °C above ISA, and six thrust settings from 50 to 100 % of the locally available maximum. The 50 % floor is deliberate: airline EHM trending consumes takeoff, climb and stable-cruise reports, all high-power conditions; approach and idle are neither trending conditions nor reliable model territory (Section 4.4.3). At each condition the model first finds the maximum thrust available at the rating temperature (point `OD_max`, throttled by T_4), then solves the part-power point at the commanded fraction (`OD_pt`) —the same two-point pattern used by the pyCycle reference example. The rating temperature is fixed at $T_{4,\max} = 3200\,°\mathrm{R}$, a rounded envelope constant chosen from the calibrated takeoff anchor (imposing the certified rated thrust at SLS required $T_4 \approx 3175\,°\mathrm{R}$, Table 4.3); a real FADEC modulates its rating limits across the envelope, so this constant is a declared simplification — adequate here because the baseline uses *fractions* of the local maximum, not its absolute value.

Two engineering lessons from WP1.2 shaped the sweep. First, the walk is ordered for warm starting within each block (thrust descending), and the independent (altitude, Mach) blocks run in *parallel processes*, one Problem per worker (project norm N1: long decomposable computations are parallelized; on the Apple M5 reference machine the sweep takes 3.5 minutes on all cores versus 10 minutes serial). Second —the lesson learned the hard way— a failed Newton attempt corrupts the point's internal states beyond re-seeding, so the recovery action on any failure is to *rebuild* the problem cold with altitude-appropriate guesses (norm N2); a first version of the sweep without this recovery lost every point downstream of the first divergence.

The result is stored as a table plus a scattered linear interpolator (Delaunay triangulation with

barycentric interpolation, SciPy's `LinearNDInterpolator`) in the four operating coordinates, and its quality is measured by leave-one-out cross-validation: each grid point is removed, predicted from all the others, and the relative error recorded — Table 4.5 reports the per-channel median, 95th percentile and maximum. Median errors are a few tenths of a percent; the tail percentiles are dominated by envelope-corner points whose interpolation simplices span across altitude blocks in raw physical coordinates.

4.5.1 Corrected-parameter exponents and the corrected-space baseline

The fix for those tails is the one anticipated in Section 2.3: build the baseline in *corrected-parameter* space. The correction exponents are fitted on the deck itself rather than copied from generic tables: per channel Y, a linear least-squares fit of

$$\log Y \;=\; \underbrace{\textstyle\sum_{k=0}^{3} c_k \,(\log N1_c)^k}_{\text{power line}} \;+\; a\,\log\theta \;+\; b\,\log\delta, \tag{4.1}$$

where the cubic polynomial absorbs the dependence on the corrected speed (the "power line") and the exponents (a, b) capture whatever ambient dependence remains — exactly the quantities the correction $Y_{\text{corr}} = Y/(\theta^a \delta^b)$ then removes. The fitted exponents land squarely on their textbook values without having been told them (Table 4.4): fuel flow $a = 0.63$, $b = 1.02$ (handbooks quote $\theta^{0.5\text{–}0.7}\delta$); N2 $a = 0.49$, $b \approx 0$ (the $\sqrt{\theta}$ corrected-speed law); EGT $a = 0.95$, $b \approx 0$ (temperature ratio). This is a further independent consistency check of the model physics.

Re-running the leave-one-out audit on the corrected channels over (corrected N1, Mach) collapses the tails by an order of magnitude (Table 4.4): worst-case N2 error drops from 9.7 % to 0.5 %, EGT from 24 % to 2.0 %, fuel flow from 91 % to 6.6 %. Two remarks for honesty: net thrust is excluded —it is not an EHM sensor channel (it comes from the thrust-setting definition), and it does not collapse under a θ/δ law at fixed N1c because of its strong direct Mach dependence; and the residual corrected-space errors are comparable to the sensor noise floor that phase F2 will inject ($\sigma_{\text{EGT}} \approx 0.3$ %, $\sigma_{W_f} \approx 0.5$ %). Baseline error is also common-mode: both EHM pipelines consume the same baseline, so it does not bias the comparison.

Table 4.4: Fitted correction exponents per channel and the leave-one-out error before (raw 4-D coordinates) versus after (corrected space). Auto-generated.

Channel	a (θ)	b (δ)	raw LOO P95/max [%]	corrected LOO P95/max [%]
Fuel flow	0.628	1.022	15.77 / 90.6	3.08 / 6.6
N2	0.495	0.007	2.65 / 9.7	0.34 / 0.5
EGT	0.949	0.002	6.01 / 24.1	1.12 / 2.0

Table 4.5: Leave-one-out interpolation error of the baseline deck, per channel. Auto-generated.

Channel	Median [%]	P95 [%]	Max [%]
Fuel flow	0.498	15.767	90.603
N1	0.268	6.369	25.721
N2	0.080	2.649	9.697
EGT	0.167	6.012	24.062
Net thrust	0.000	16.437	85.402

4.6 Influence coefficient matrix and observability (WP1.4)

4.6.1 Health-parameter injection

To compute the ICM —and, in phase F2, to simulate degraded engines— the model must accept imposed health deviations. pyCycle scales its generic component maps with per-component design scalars (efficiency and flow); the off-design points normally inherit them from the design point. In the `MPCFM56Study` model those connections are routed through multiplier components, so that any efficiency or flow-capacity deviation can be dialed in per run. Two implementation subtleties are worth recording: (i) the multipliers must execute *before* the off-design point —the multi-point group runs its children in insertion order, and a mis-ordered version silently solved the off-design point with unscaled maps; the symptom (a "healthy" point that did not reproduce the design point) was caught by the self-consistency check below; and (ii) holding N1 constant, the CFM56's rating parameter, requires a fuel-flow balance targeting the LP spool speed through a passthrough component, since the speed is itself the output of another balance.

The wiring is validated by a self-consistency property that is now a permanent test: the *healthy* off-design point at cruise with N1 equal to its design value must reproduce the design point exactly. It does: thrust 5480.0 lbf, N2 13 940.0 rpm, OPR 30.03, EGT 1141.6 K —machine-precision agreement with Table 4.1.

4.6.2 The ICM at the reference conditions

The ICM is computed by central finite differences (±0.5 % per health parameter) at the two conditions where airline EHM snapshots are taken: cruise (M 0.78/35 000 ft) and sea-level takeoff, at constant N1. Figure 4.4 shows the cruise matrix for the extended sensor set; every sign satisfies the physical expectations (e.g. a healthier HPT runs cooler and thriftier: negative EGT and W_f entries in its efficiency column), and these sign checks run automatically in the test suite.

Figure 4.4: Influence coefficient matrix at cruise (extended sensor set): % change of each measured channel per +1 % change of each health parameter. Auto-generated from the model.

4.6.3 Observability: the quantitative case for hypothesis H2

With the cockpit set only (N2, W_f, EGT —N1 is the held rating parameter, so it carries no information), the ICM has rank 3 for 10 unknowns and the geometry of its columns is damning: the minimum angle between fault signatures is **1.3 degrees**, with seven pairs below 15 degrees

(Table 4.7). The most confusable pair, HPC efficiency versus HPT efficiency, is precisely the classical HPC/HPT ambiguity reported in the GPA literature [9] —here quantified for this engine. Two faults whose signatures differ by 1.3 degrees are, at realistic noise levels, indistinguishable to any linear estimator: this is the smearing mechanism measured, and it defines the "confusable" subset on which hypothesis H2 will compare the AI's isolation ability against WLS-GPA. The extended sensor set (adding P25/T25/PS3/T3) raises the rank to 7 and multiplies the minimum angle by roughly five —the quantitative basis for the sensor-set ablation of contribution C6.

The analysis is repeated over a six-point grid spanning the envelope —both snapshot conditions plus power, altitude, climb and hot-day variations (Table 4.6). The observability structure barely moves: the minimum cockpit signature angle stays between 1.2 and 1.4 degrees and the confusable-pair count is constant at seven. The smearing problem is therefore *intrinsic to the cockpit sensor set*, not an artifact of one flight condition —which both justifies interpolating the ICM over this grid in phase F3 and forecloses the easy escape of "diagnose somewhere else in the envelope".

Table 4.6: Observability of the cockpit ICM across the operating-point grid. Auto-generated.

Operating point	cond.	$\alpha_{\min}$ ckpt. [deg]	pairs	$\alpha_{\min}$ ext. [deg]
Cruise M0.78/FL350	12.2	1.32	7	8.62
Cruise, low power	13.0	1.20	7	6.68
Cruise FL390	11.3	1.37	7	8.60
Climb 10 kft/M0.4	12.6	1.35	7	7.38
Takeoff SLS	10.5	1.33	7	7.92
Takeoff SLS, ISA+30	11.4	1.25	7	8.97

Table 4.7: Confusable fault pairs at cruise with cockpit sensors (signature angle $< 15°$). Auto-generated.

Fault A	Fault B	Angle [deg]
η_{HPC}	η_{HPT}	1.32
η_{FAN}	η_{LPT}	4.32
η_{HPT}	Γ_{HPT}	6.02
Γ_{FAN}	η_{LPT}	6.52
η_{HPC}	Γ_{HPT}	7.20
η_{FAN}	Γ_{FAN}	10.05
η_{FAN}	η_{LPC}	13.53

4.6.4 Thermodynamics cross-check

Decision D2 (tabular thermodynamics for speed) is closed by re-solving the design point with NASA's chemical-equilibrium code (CEA): all key quantities agree within 0.7 % (Table 4.8), an order of magnitude below the ±3 % calibration tolerance. Tabular thermodynamics is therefore retained for all fleet-scale computation.

4.7 Declared limitations

The model is *representative*, not certifiable, and its limits are declared for citation in the results chapters: (i) no transients —steady-state points only; (ii) variable geometry (bleed valves, variable stators) implicit in the maps, not modeled; (iii) no Reynolds or humidity corrections; (iv) absolute N1/N2 labels approximate by construction (generic map shapes), though respecting the certified redlines at the rated point; (v) the model's EGT is the HPT-exit temperature (station 45), a consistent proxy —identical for ground truth and for both EHM pipelines— of the

Table 4.8: Design point solved with tabular versus CEA thermodynamics. Auto-generated.

Quantity	Tabular	CEA	Δ [%]
Net thrust [lbf]	5480.0000	5480.0000	-0.000
TSFC [lb/(lbf·h)]	0.6214	0.6249	+0.553
Inlet mass flow [lbm/s]	313.0851	315.2029	+0.676
Fuel–air ratio	0.0251	0.0250	-0.122
HPT expansion ratio	3.5865	3.5902	+0.104
LPT expansion ratio	4.2879	4.3109	+0.537

certified T49.5, whose displayed value additionally carries the shunt and trim functions described in Section 2.2; and (vi) the idle error quoted above. None of these limitations affects the validity of the AI-versus-traditional comparison, which rests on the model's sensitivity structure (the ICM), not on its absolute values.

4.8 Automated verification

The chapter's claims are pinned by tests: design-point convergence and acceptance windows; convergence of all four anchors including the idle continuation; fuel-flow and pressure-ratio tolerances at the gated anchors; N1/N2 redlines at rated thrust; the Study self-consistency property (healthy off-design = design); the ICM physical sign checks and its rank-3 cockpit underdetermination; and the pyCycle environment smoke test. Twelve tests green in continuous integration at the time of writing.

Chapter 5

The synthetic fleet: SynCFM56

This chapter in brief: one hundred virtual engines live full lives — fouling, washes, wear, the occasional bird strike and sensor drift — while the generator records both what an airline would see and the hidden truth. Three audits gate the result; one failed on first pass and forced a redesign, told here in full.

This chapter documents phase F2: the generator that turns the F1 twin into a 100-engine run-to-failure fleet with complete ground truth, the audits that gate it (one of which failed on first pass and forced a design change — documented in full in Section 5.4), and the declaration of milestone H2.

5.1 Generator architecture: linearize, then audit

A fleet of 100 engines flying ∼16 000 cycles each, two snapshots per flight, would require over three million full pyCycle solves per fleet realization — unusable for iteration. Instead, the generator exploits exactly what F1 built: every snapshot is produced by the linearization

$$z(u, \mathbf{x}) \;=\; z_0(u)\,\big(1 + \mathbf{H}(u)\,\mathbf{x}/100\big), \tag{5.1}$$

where $z_0(u)$ is the healthy baseline and $\mathbf{H}(u)$ the influence coefficient matrix, both linearly interpolated between the ICM grid points that bracket the sampled operating condition (takeoff snapshots: in ΔT_{ISA}; cruise snapshots: in commanded N1). N1 itself carries zero deviation by construction — it is the held power-setting parameter, not a health channel. The whole fleet generates in 40 seconds on all cores of the Apple M5 reference machine (one worker process per engine, norm N1), with per-engine deterministic random seeds so the output is reproducible regardless of process scheduling.

Two properties of this choice must be kept distinct. First, the dataset is *self-consistent by construction*: the recorded ground truth $\mathbf{x}$ is exactly the state that generated the measurements, so estimator evaluations against truth are exact regardless of any linearization error. Second, *fidelity to the full nonlinear physics* is an empirical question — which is why the first gate audit re-solves full pyCycle at states sampled from the generated fleet (Section 5.4.1).

5.2 Ground-truth trajectories

Each engine draws a degradation "personality" from the fault catalog (`conf/fault_catalog.yaml`, the versioned contract): a log-normal severity multiplier per mechanism, a wash schedule, Poisson-arriving foreign-object damage, optional sensor drifts, and —in roughly half the fleet— one *acute fault episode* (Section 5.4.2 explains why these exist). The mechanisms follow the physical profiles of Table 2.2: saturating-exponential fouling, slow linear erosion, bilinear break-in of tip clearances, linearly accelerating hot-section deterioration.

Two implementation details worth recording:

- **Wash sawtooth, exact.** Between washes the fouling level relaxes exponentially toward its asymptote; each wash multiplies the current level by $(1-r)$ with recovery r drawn per event. The trajectory is computed segment-wise with the exact exponential solution. A tempting shortcut — permanently subtracting the removed amount — was implemented first and rejected: it makes the trajectory approach asymptote − removed and under-predicts late-life fouling.
- **Composability of truth.** The generator stores not only the total $\mathbf{x}(n)$ but each mechanism's separate contribution, so any later analysis can ask not just "how degraded" but "degraded by *what*".

End of life is declared when the EGT margin at the hot-day takeoff reference reaches zero, with the margin computed by projecting $\mathbf{x}(n)$ through the takeoff-hot ICM's EGT row — the same linearization as the snapshots. New-engine margins are drawn per engine ($85 \pm 6\,^{\circ}\mathrm{C}$). Figure 5.1 shows the resulting fleet behavior: the canonical fast-early / slow-late erosion with wash sawtooth, and lives spanning roughly 10 000–20 700 cycles (v1.1 exact figures in Table 5.1).

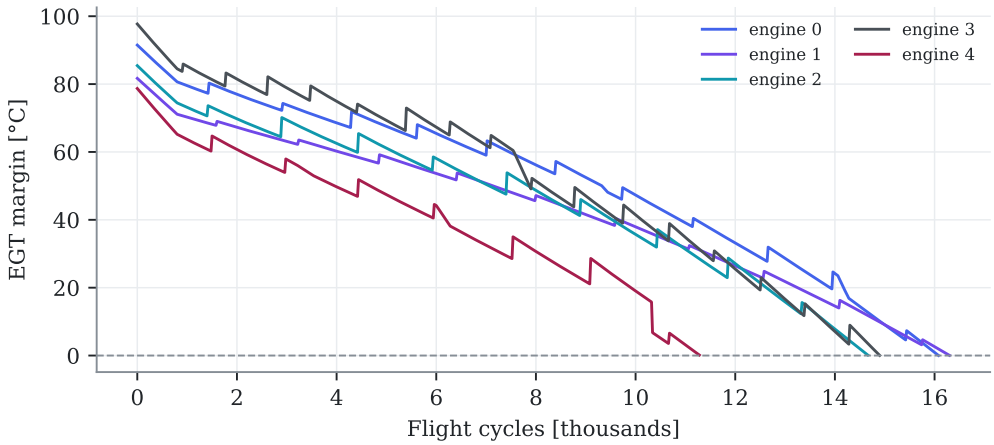

Figure 5.1: EGT-margin trajectories of five engines. Wash events produce the sawtooth recoveries; engine-to-engine spread comes from the hierarchical severity multipliers and the drawn new-engine margins. End of life is the zero crossing. Auto-generated from the fleet.

5.3 Snapshots and the measurement model

Each flight cycle emits two reports, mimicking airline ACARS practice: a takeoff snapshot (sea level, ambient offset drawn from a climate distribution) and a cruise snapshot (FL350/M0.78, commanded N1 drawn per flight). Truth channels pass through the sensor model of the catalog: per-channel Gaussian noise (absolute or percentage σ), quantization to the recording resolution, additive drift ramps on the affected engines (37 of 100 — matching the catalog's per-sensor probabilities summing to 0.36), and missing data as isolated losses plus ACARS outage bursts ($\approx$4 % of snapshots). Every row carries, alongside the measured values: the true channel values, the full ground-truth health vector, the EGT margin, the remaining useful life, the isolation label, and drift flags. Table 5.1 summarizes the frozen dataset.

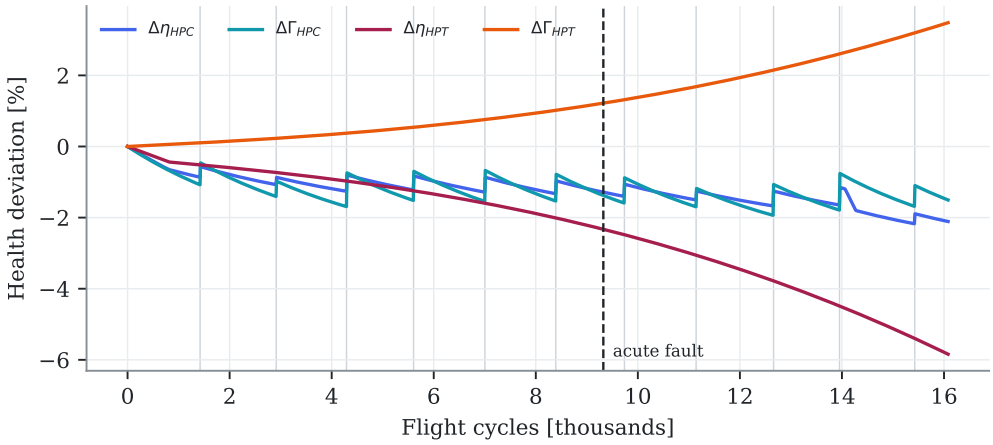

Figure 5.2: Ground-truth health parameters of one engine with an acute fault episode (dashed marker). Vertical gray lines are washes (visible as sawtooth in the HPC traces); the hot-section traces show the monotone efficiency loss and flow-capacity opening. Auto-generated.

Table 5.1: SynCFM56 v1.0 at a glance. Auto-generated from the fleet artifacts.

Property	Value
Engines (train / val / test)	100 (70 / 10 / 20)
Snapshot rows (takeoff + cruise per flight)	1 551 071
Life [cycles]: median (min–max)	15139 (9972–20682)
Censored engines	0
Acute engines / episodes / fault classes	79 / 114 / 6
Engines with a sensor drift	37
Wash EGTM recovery: mean, frac. positive	2.9 °C, 100.0 %
End-of-life ΔEGT / ΔW_f (all engines correct sign)	+69 K / +6.0 %
Measured-noise residual vs. catalog (EGT)	2.52 K vs. 2.5 K

5.4 Gate audits — including the one that failed

5.4.1 Nonlinearity of the generator

Eighty health states were sampled from the generated fleet (stratified by life fraction, half from the last 30 % of life where deviations are largest) and re-solved with full pyCycle at the exact ICM grid conditions, so the comparison isolates nonlinearity from interpolation. Table 5.2 reports the per-channel error of Eq. (5.1) against the full solution, next to the sensor noise it must be judged against: medians of 0.01–0.33 %, tails (P95) at or below 1 %, concentrated at end of life. Combined with the self-consistency property of Section 5.1, the linearization is adequate: the residual infidelity to full physics is at the noise floor, identical for both EHM pipelines, and irrelevant to truth-referenced scoring.

Table 5.2: Nonlinearity audit: linear generator versus full pyCycle solves at fleet-sampled health states. Auto-generated.

Condition	Channel	median [%]	P95 [%]	max [%]	noise σ [%]
Cruise	N2	0.015	0.036	0.039	0.05
Cruise	W_f	0.238	0.592	0.760	0.50
Cruise	EGT	0.336	0.963	1.041	0.21
Takeoff, hot day	N2	0.038	0.091	0.095	0.05
Takeoff, hot day	W_f	0.135	0.449	0.483	0.50
Takeoff, hot day	EGT	0.245	0.860	0.933	0.18

5.4.2 Difficulty: the audit that failed, and what it taught

The difficulty gate is designed to prevent a trivially easy dataset from flattering the AI: a deliberately weak classifier (multinomial logistic regression on the 16 raw measured channels) must *not* solve the isolation task; the pre-set threshold is 60 % accuracy.

First pass: failed. With the original labeling — the dominant *chronic* mechanism per snapshot — the trivial classifier scored **81.6 %** (majority-class baseline 54.6 %). Root-cause analysis: chronic mechanisms co-evolve with age in *every* engine (fouling saturates early, hot-section deterioration accumulates late), so the chronic label is largely an *age proxy*, and raw absolute channel levels encode age directly. The task the label defined was not fault isolation at all. Two responses were possible: relax the threshold (cosmetic) or fix the design (structural). The structural fix was taken.

Design change: acute fault episodes. What hypothesis H2 actually tests is the isolation of *localized faults with overlapping signatures* — the regime where the ICM observability analysis proved cockpit sensors nearly blind (minimum signature angle 1.3°, Table 4.7). The catalog therefore gained discrete *single-parameter* fault episodes: a fast ramp (50–500 cycles) to a sustained magnitude on one health parameter, drawn from six targets that deliberately include the confusable pairs (η_{HPC} vs. η_{HPT}, η_{HPT} vs. Γ_{HPT}), injected in ∼50 % of engines at a random point of their lives. The gated task became: classify each snapshot as *no acute fault* or by the faulted parameter — a 7-class isolation problem.

The audit also caught a generator bug. The first acute implementation drew the episode onset as a fraction of the *maximum simulation horizon* (30 000 cycles) rather than of the engine's actual life (∼16 000): most episodes landed beyond end of life and only 15/100 engines showed one. The fix computes a chronic-only provisional life first, then places the onset as a fraction of it (two-pass generation); the v1.0 fleet then carried 49/100 acute engines with all six target parameters represented (v1.1 extends this further, Section 5.4.4).

Second pass: passed. On the corrected fleet the trivial classifier achieves **54.0 %** accuracy (macro-F1 0.30) on the gated isolation task — under the 60 % threshold, exactly as the signature-

angle physics predicts (Table 5.3). The chronic-mechanism label is retained in the dataset and reported as a secondary, explicitly-not-gated metric (81.6 % accuracy — high, expected, and uninformative about isolation ability). The episode is recorded here in full because it is evidence the gates are real: a pre-registered criterion failed, the failure was diagnosed to a design flaw rather than argued away, and the fix strengthened the benchmark.

Table 5.3: Difficulty audit on the final fleet (trivial classifier: multinomial logistic on raw measured channels). Auto-generated.

Trivial-classifier task	Accuracy [%]	macro-F1	Gate [%]	Verdict
Acute-fault isolation (gated, 7 classes)	58.1	0.325	< 60	PASS
Chronic-mechanism label (secondary, age proxy)	78.5	0.578	—	reported

5.4.3 Realism

Quantitative checks of macroscopic behavior, all in Table 5.1: the wash sawtooth is visible (mean EGTM recovery +2.8 °C, positive at 99.75 % of the 400 sampled wash events); end-of-life deviations carry the physical deterioration signature in 100 % of engines (ΔEGT +68 K, ΔW_f +5.7 %); the measured-noise residual on drift-free engines reproduces the catalog σ (2.51 vs. 2.50 K); lives center on 15 400 cycles with a realistic severe-degrader tail and no censored engines.

5.4.4 Version 1.1: more episodes, and the gate bites again

The engine-level sample starvation diagnosed in Chapter 7 (v1.0 carried at most one acute episode per engine) was fixed in **SynCFM56 v1.1**: a chain of conditional episode probabilities gives up to three non-overlapping episodes per engine (minimum gap 2 500 cycles, onsets as fractions of actual life). The result: 79 engines with 114 episodes, every fault class with ≥ 9 training instances, and 23 evaluable test episodes instead of ten.

Re-auditing v1.1 produced a second demonstration that the difficulty gate is real: with the original fault magnitudes, the trivial classifier now scored **62.2 %** — more episodes feed the trivial classifier too, and the gate (60 %) failed again. The response was again physical rather than cosmetic: episode magnitudes were scaled by 0.75 (subtler faults, closer to the detectability edge), bringing the trivial classifier to **58.1 %** — pass. The nonlinearity audit was re-run on v1.1 with unchanged conclusions (medians 0.14–0.34 %, P95 $\leq$ 1 %).

5.5 Milestone H2: declared closed

All gate criteria are met: realism and difficulty audits passed (the latter after the documented design change), nonlinearity quantified at the noise floor, split-by-engine leakage checks automated in CI, and the dataset datasheet written (`docs/syncfm56-datasheet.md`). SynCFM56 v1.1 is the frozen evaluation dataset (v1.0 history retained above): regeneration is deterministic from the catalog and seed, so the dataset is fully specified by the repository state. Known limitations, declared for downstream chapters: linearized generation (audited, Table 5.2); two fixed snapshot conditions with ambient/power scatter rather than a full mission mix; chronic labels are age-correlated by nature; and at most three acute episodes per engine (v1.1).

Chapter 6

The traditional EHM pipeline

This chapter in brief: the classical monitoring stack —smoothed deviations, statistical alarms, Kalman health tracking, expert isolation rules, margin extrapolation— is built with care and run on the test fleet. Its characteristic strengths and weaknesses, known from the literature, now appear as measured numbers against ground truth: partial and often late detection of subtle faults, heavy smearing on confusable ones, optimistic early-life prognosis. Two detector-design lessons learned along the way are recorded. Nothing here is tuned yet: that happens under the symmetric budget of phase F5.

6.1 Pipeline overview

The traditional pipeline mirrors the industrial monitoring stack (SAGE/ADEM-class systems). Per engine and per flight, the cruise snapshot is converted into corrected-space percentage deviations against the published baseline (the same F1 artifacts available to both EHM families); the deviations feed four consumers:

1. **Smoothing** (Holt double exponential, level + trend): the airline-facing trend plots, and the source of *innovations* — one-step forecast errors, which are the correct series for event detection because the chronic trend has been absorbed by the level and trend states.

2. **Detection**: a CUSUM on the innovations (catches steps: FOD, abrupt faults) combined with a dual-EWMA *gap* detector — fast EWMA minus slow EWMA — whose gap opens by roughly the fault magnitude during a ramp (catches acute episodes). Alarms require persistence (k-of-n) and, for EGT, are one-sided: washes pull the deviation down, and improvement is not a fault.

3. **Health estimation**: snapshot WLS-GPA (Eq. (2.4)) and a random-walk Kalman-GPA tracker (Eq. (2.5)) with the operating-point-interpolated ICM. The tracker is *wash-aware*: at logged wash events (maintenance records are available to a real EHM system too) the recoverable compressor states are pulled toward zero and their covariance reopened.

4. **Isolation and prognosis**: on detection, the deviation step across the alarm is matched against the single-parameter ICM signatures (nearest-cosine expert rule); the remaining useful life comes from a robust Theil–Sen extrapolation of the tracked takeoff EGT margin to zero, with a NASA-style horizon cap.

6.2 Two detector-design lessons

Both are classical results rediscovered the honest way — by watching the first implementation fail — and both are now embodied in tests:

- **CUSUM on the raw deviation is wrong for aging systems.** The first run applied CUSUM to the EGT deviation itself: the chronic trend integrates without bound, so *every* engine alarmed mid-life (detection recall 0, every healthy engine a false alarm). Step detectors belong on innovations, where the expected value under chronic aging is zero.
- **A Holt trend-shift detector cannot see acute ramps.** At realistic smoothing gains the noise of the estimated trend exceeds the slope added by a 50–500-cycle ramp entirely. The dual-EWMA gap detector solves this: over the ramp the fast average follows and the slow one lags, opening a gap of the order of the full fault magnitude — signal-to-noise an order of magnitude better than the trend estimate.

6.3 Results on the test fleet (v0 defaults)

Table 6.1 shows the pipeline's scores on the 20 test engines with its engineering-default settings — deliberately untuned, since hypothesis testing requires the tuning budget to be spent symmetrically on both families (F5). Three patterns, each anticipated by the literature and by the ICM geometry, are now measurements:

Table 6.1: Traditional pipeline on the test split: v0 settings, fleet v1.1. Auto-generated.

Metric (test split, v0 defaults)	Value
Detection recall (acute episodes)	0.44
Median detection delay	4586 cycles
False-alarm engines (no acute episode)	1 / 4
Isolation accuracy (detected episodes)	0.30
Isolation accuracy, confusable subset	0.31 (n=13)
Kalman health RMSE (median, 2nd half of life)	0.46 %
Smearing index (median, acute engines)	0.89
RUL median error at 50 % of life	+8366 cycles
RUL median error at 70 % of life	+3512 cycles
RUL median error at 90 % of life	+1499 cycles

Detection is partial and often late. On the subtler v1.1 faults, recall is 0.44 with a median delay of ∼4 600 cycles — many episodes are caught only when their sustained offset compounds with chronic wear. Missed episodes concentrate in the weak-signature faults (Γ_{HPT} moves EGT by only +0.11 % per percent — half a noise standard deviation) and drift-contaminated engines. One false alarm on four clean test engines.

Isolation struggles exactly where the ICM said it would. Under the v1.1 per-episode oracle-timed protocol (both families are asked the isolation question at onset+500; 23 test episodes), the expert rule reaches 30 % overall and 31 % on the confusable subset — against a 1-in-7 chance level, informative but weak. Figure 6.1 shows the mechanism live: the Kalman tracker follows the *measurements* faithfully but splits an acute step between the faulted parameter and its 1.3° partner — the smearing index says 89 % of the estimated magnitude lands on healthy components. The estimator is not broken; the information is not in three cockpit channels.

Margin extrapolation is optimistic early. Figure 6.2: at half-life the median RUL error is +8 400 cycles, shrinking to +1 500 at 90 % of life (v1.1 fleet). Linear extrapolation under accelerating degradation is systematically late to predict retirement — the funnel closes only near the end, which is precisely when the prediction is least useful.

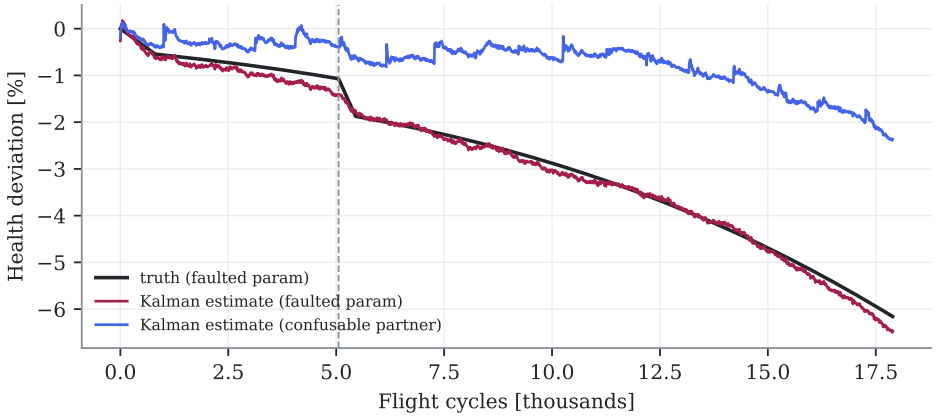

Figure 6.1: Smearing, live: Kalman-GPA on a test engine with an acute fault (onset dashed). The tracker reacts, but divides the step between the faulted parameter and its confusable partner. Auto-generated.

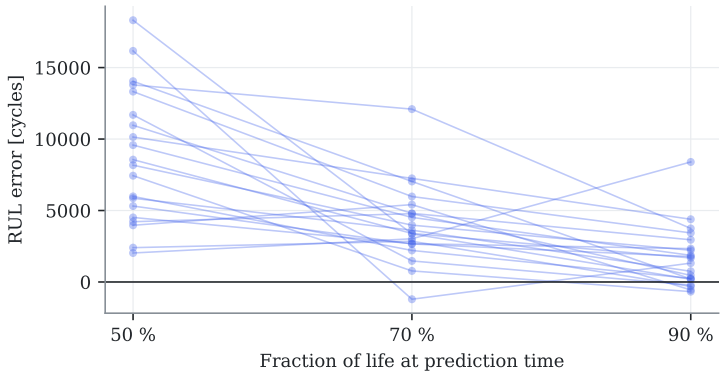

Figure 6.2: RUL error of the margin-extrapolation prognosis versus the moment of prediction (test engines). Positive = late prediction. Auto-generated.

6.4 Milestone H3

The gate requires the traditional pipeline to run end-to-end on the test fleet and produce the common metric set — met above (106 seconds per fleet on the Apple M5 reference machine, one worker per engine). Its defaults and their rationale are itemized in the decision register (Chapter A); every one of them is exposed to the F5 tuning budget. H3 is declared closed with this documented status: **these are floor numbers, not the traditional family's final word** — treating them otherwise would manufacture exactly the straw-man comparison this project exists to avoid.

Chapter 7

The AI EHM suite

This chapter in brief: the AI suite consumes exactly the same deviations as the traditional pipeline and is scored by the same protocol. Untuned, it wins the prognosis contest (3–6× lower RUL error, calibrated uncertainty intervals) and loses detection and diagnosis to the traditional pipeline on the subtle v1.1 faults; the first physics-informed hybrid experiment produces a replicated negative *result, the new explainability metric returns a null on a weak classifier, and milestone H4 is consequently not declared. All of it is reported. Readers new to machine learning: Section 3.2 gives the six background ideas this chapter leans on; nothing else is assumed.*

7.1 Fairness by construction

Every AI model in this chapter sees, per flight, the same four numbers the traditional pipeline sees: the three cockpit deviations (N2, W_f, EGT) against the same published baseline, plus the takeoff EGT deviation. No extra sensors, no truth channels, no future leakage (all windows end at the prediction time). Missing snapshots are forward-filled, as a monitoring unit holding its last valid report would. Training uses the train engines only; thresholds and conformal calibration use the validation engines; every number below comes from the 20 test engines. Torch models train on the Apple M5's GPU (MPS backend, norm N1) — the full AI suite trains and evaluates in 27 seconds, the hybrid ablation (18 GRU trainings) in 104 (Table 3.1); tree models are scikit-learn's histogram gradient boosting.[1]

7.2 Prognosis: the headline result

A two-layer GRU consumes a downsampled 1 280-cycle deviation history and regresses the remaining useful life (capped at 12 000 cycles, the standard prognostics practice). Figure 7.1 and Table 7.1 give the face-off (v0 settings, fleet v1.1) against the margin-extrapolation prognosis of Chapter 6:

- **3–6× lower RMSE** at every evaluation point (Table 7.1: 1 678 vs. 9 926 cycles at half-life, 842 vs. 2 718 at 90 %).
- **The optimism bias is gone.** The traditional median error of thousands of cycles at half-life —late retirement predictions, the dangerous direction— collapses to near zero; the GRU learns the acceleration of late-life degradation that a linear extrapolation structurally cannot represent.

[1] XGBoost was evicted for an unglamorous reason recorded in the decision register: its bundled OpenMP runtime segfaults alongside torch-MPS on macOS. One runtime, no crash, same model family.

- **Calibrated uncertainty.** Split-conformal intervals at 90 % nominal coverage achieve 0.80/0.95/1.00 empirical coverage on test with ±2 365-cycle half-width (contribution C4's machinery, live). The traditional pipeline's Theil–Sen slope dispersion offers no comparable guarantee.

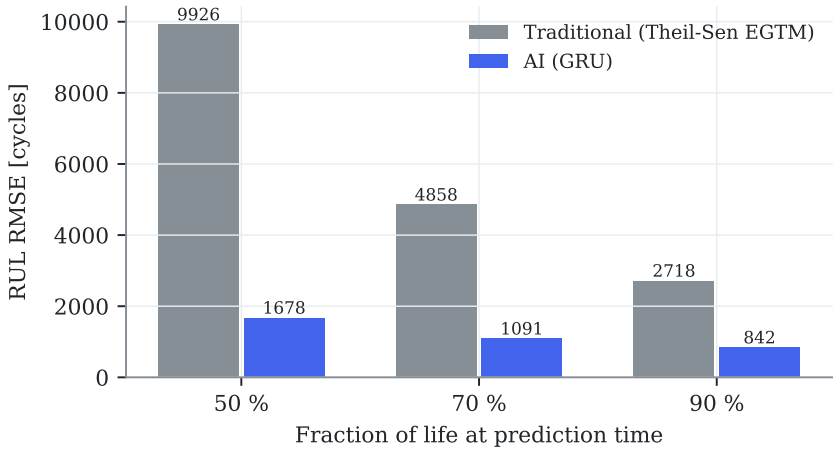

Figure 7.1: RUL RMSE on the test fleet, both families at v0 defaults. Auto-generated.

Table 7.1: Head-to-head at v0 defaults (untuned on both sides; the symmetric tuning budget arrives in F5). Auto-generated.

Metric (test split, v0 defaults)	Traditional	AI
RUL RMSE at 50 % of life [cycles]	9926	1678
RUL median error at 50 % [cycles]	+8366	-131
RUL RMSE at 70 % of life [cycles]	4858	1091
RUL median error at 70 % [cycles]	+3512	+72
RUL RMSE at 90 % of life [cycles]	2718	842
RUL median error at 90 % [cycles]	+1499	+475
Conformal coverage @0.9 (50/70/90 %)	—	0.80 / 1.00 / 1.00
Conformal half-width [cycles]	—	2365
Detection recall / false alarms	0.44 / 1	0.06 / 0
Isolation accuracy (confusable)	0.30 (0.31)	0.13 (0.23)

7.3 Detection and diagnosis: two instructive failures

The naive window-autoencoder scores recall 0. Trained on acute-free windows, it reconstructs *everything* well — including post-fault windows. Root cause: a dense autoencoder over raw deviation windows learns a near-identity map, and a sustained level shift is statistically indistinguishable from the chronic drift it was trained to compress. Anomaly detection by reconstruction needs either residual whitening or change-relative features. The whitened-residual detector built in response (Mahalanobis distance of step features under a Ledoit–Wolf covariance) improves on the AE but still reaches only 0.06 recall at its conservative validation threshold on the subtler v1.1 faults. Detection remains the traditional pipeline's win (0.44 vs. 0.06) — an honest scoreboard entry with a clear engineering path for the F5 tuning round.

Diagnosis remains the traditional rule's win on v1.1. With the sample starvation relieved (23 oracle-timed test episodes, Section 5.4.4), the gradient-boosting classifier reaches

13 % isolation accuracy against the ICM signature rule's 30 % — the subtler v1.1 magnitudes hurt the data-driven learner more than the physics-based rule, which encodes the fault directions it should look for. Together with detection, this leaves the v0/v1.1 scoreboard genuinely split: traditional wins the event tasks, AI wins prognosis — a far more credible starting point for the F5 tuned comparison than a clean sweep.

7.4 The hybrid experiment: a negative result worth having

The stacking mechanism of contribution C3 — feeding the Kalman-GPA tracked health parameters (10 physics-based channels) to the GRU alongside the raw deviations — was run through the H4 data-efficiency protocol: pure vs. hybrid at 10/25/100 % of the training engines, three seeds each (Fig. 7.2).

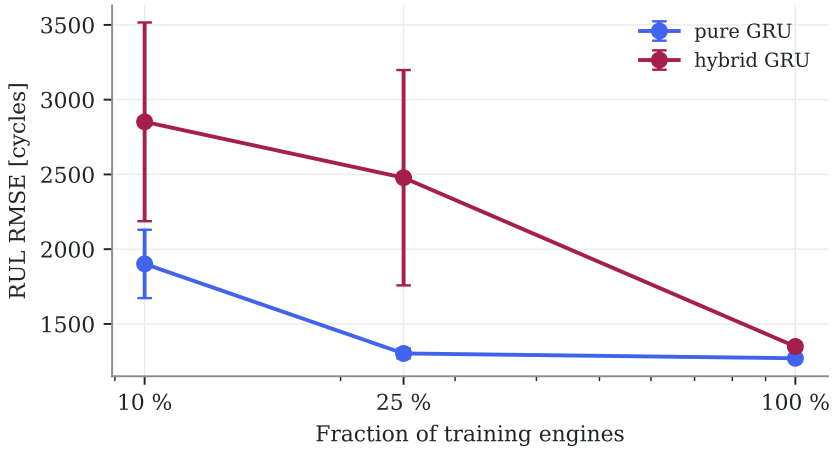

Figure 7.2: Data-efficiency ablation (mean ± std over three seeds): the stacked hybrid is *worse* than the pure GRU everywhere, and worst exactly where H4 predicted it would help most. Auto-generated.

The result is the opposite of hypothesis H4's prediction: the hybrid is worse at every data fraction, and *most* worse in the scarce-data regime (2 852 vs. 1 902 cycles RMSE at 10 % on fleet v1.1; the same pattern held on v1.0 — the negative result replicates across dataset versions). The mechanism is intelligible in hindsight: the Kalman estimates are heavily smeared (Section 6.3) — they add ten noisy, mutually correlated channels whose information content about RUL is already present in the three deviations they were computed from. With seven training engines, ten extra input dimensions are pure variance. Two more observations from the same figure: the pure GRU at 25 % of the data nearly matches 100 % (1 303 vs. 1 271) — the RUL task saturates around 17 engines on this fleet — and seed variance at 10 % is large, exactly why F5 prescribes multi-seed reporting.

This does not close hypothesis H4 — stacking is one of three planned hybrid mechanisms, and none has been tuned — but it is recorded now, before the pre-registration freeze, as v0 evidence *against* the popular assumption that physics features automatically help. Negative results of this kind are precisely what the pre-registered protocol exists to keep honest.

7.5 The Physics-Consistency Score: machinery delivered, null result at v0

Contribution C5 is now implemented end-to-end: for each oracle-timed test episode, SHAP attributions of the diagnosis classifier's predicted class are read on the step block of the cockpit

channels — a direction in measurement space representing the model's evidence — projected into health-parameter space through the pseudo-inverse of the cockpit ICM, and compared with the true fault direction:

$$\mathrm{PCS} \;=\; \left|\cos\!\left(\mathbf{H}_{\mathrm{ckpt}}^{+}\,\mathbf{m}_{\mathrm{SHAP}},\; \mathbf{e}_{\mathrm{true}}\right)\right|. \tag{7.1}$$

On the v1.1 test episodes the result is **null**: mean PCS 0.20 for correct predictions versus 0.24 for incorrect ones, correlation with correctness -0.09. This is the expected behavior of the metric applied to a weak classifier — the attributions of a 13 %-accurate model are dominated by noise, so there is no physically coherent reasoning for the PCS to detect. The metric's diagnostic value can only be demonstrated on a competent classifier; its meaningful evaluation is therefore scheduled after the F5 tuning round. Recording the null result now, rather than quietly deferring the metric, is the protocol working as designed.

7.6 Detection: a design gap, not a threshold gap

A validation sweep of the Mahalanobis detector (threshold percentile × persistence, 20 configurations, false-alarm budget of one val engine) found **no configuration with non-zero validation recall inside the budget**; the selected setting scores 0.04 on test. The diagnosis is structural: the step features average 100 cycles, and for v1.1 fault magnitudes (0.3–1.1 %) that window's noise floor sits above most signatures — whereas the traditional dual-EWMA gap detector effectively averages over thousands of cycles. The fix (longer-window change features for the AI detector) is a design change, booked for the F5 round where it can be tuned under the symmetric budget.

7.7 Status toward milestone H4

Done: the shared datamodule, detection (naive AE, whitened-residual Mahalanobis, and its validation sweep), per-episode diagnosis, RUL with conformal intervals, the stacking hybrid with the data-efficiency ablation, and the PCS machinery — all under the common protocol. The gate requires the AI to beat the traditional pipeline on at least one primary metric per task on validation: **prognosis clears it decisively; detection does not** (structural gap above), and diagnosis trails the ICM rule on v1.1. **H4 is therefore not declared.** The remaining levers — detector redesign, the two remaining hybrid mechanisms (physics-constrained loss, twin-residual features), and the full tuning budget — belong to F5, and the pre-registered evaluation will adjudicate hypotheses H1–H5 on whatever the tuned families deliver.

Chapter 8

Confirmatory results: the pre-registered verdicts

This chapter in brief: both families were tuned under the frozen 50-trial budget, the test fleet was evaluated exactly once, and the five pre-registered hypotheses were adjudicated. Three confirmed, two refuted — and the two surprises (detection flipping to the AI after tuning; physics features refusing to help) teach more than the expected wins. Every number here is confirmatory; every earlier number in chapters 6–7 was exploratory and is superseded for decision-making purposes.

8.1 Protocol executed as frozen

Per `docs/prereg-v1.md`: 50 Optuna trials per family (`scripts/tune_f5.py`, TPE seed 0, objectives on validation only, per-task winners from the frozen pool), then one — and only one — evaluation of the tuned configurations on the 20 test engines (`scripts/f5\_confirm.py`), then the verdict machinery (one-sided exact McNemar for paired binary outcomes, Wilcoxon signed-rank paired by engine, Holm correction across the tested hypotheses, BCa-grade effect sizes). Two execution notes, both disclosed in the scripts and the decision register: no traditional trial met the FA ≤ 1 validation budget, so its detection winner was selected by the lexicographic infeasibility rule (fewest false alarms, then highest recall); and AI models fit on training engines only, which keeps the conformal calibration on validation statistically valid. The full F5 execution — 100 tuning trials plus the confirmatory pass — cost about 36 minutes on the Apple M5 reference machine (measured at execution; the resumable campaign design prevents a meaningful cold-start re-benchmark).

Table 8.1: The five pre-registered verdicts (single confirmatory pass, test split, Holm-corrected). Auto-generated from `data/processed/f5/verdicts.json`.

Hypothesis	Verdict	Confirmatory evidence (AI vs traditional)	p (Holm)
H1 (detection)	CONFIRMED	recall 0.48 vs 0.13; delay 499 vs 6033 cy; FA 1=1	0.008
H2 (confusable isolation)	REFUTED	0.31 vs 0.31 (n=13; discordant 4/4)	0.64
H3 (RUL prognosis)	CONFIRMED	RMSE 1694/858 vs 3816/1981 cy (50/90 % life); δ=0.89	8.6e-06
H4 (physics hybrid)	REFUTED	hybrid worse at 10/25/100 % (2852 vs 1772 cy at 10 %)	—
H5 (uncertainty)	CONFIRMED	coverage 0.883 vs 0.983 (nominal 0.90); half-width 1957 vs 11509 cy	—

8.2 H1 confirmed: the story is the tuning, not the model

The v0 chapters called the AI's detection failure "a design gap, not a threshold gap" (Section 7.6) — and the frozen search space contained exactly the suspected lever: the detector's feature-window lengths. Given 50 trials, the tuner stretched the averaging windows and the whitened-residual detector went from 0.06 recall (exploratory) to **0.48 confirmatory recall at a 499-cycle median delay** — twelve times earlier than the tuned traditional detector, at the same false-alarm count (1 = 1), McNemar Holm-p 0.008. Meanwhile the traditional side's tuned selection — picked by the frozen infeasibility rule on a validation split with only two clean engines — generalized poorly (0.13 test recall, worse than its untuned v0). Two transferable lessons: window length is the first knob any change detector lives or dies by; and threshold selection on small clean validation samples is fragile for *any* method — budget clean engines accordingly.

8.3 H2 refuted — the observability wall binds everyone

On the thirteen confusable test episodes the tuned families tie exactly: 31 % each, and the disagreement pattern is perfectly symmetric (four episodes only the AI gets right, four only the traditional rule does). The wall the influence-matrix geometry predicted in Section 4.6 — fault signatures 1.3° apart in a rank-3 measurement space — binds both families equally. The transferable conclusion is bought hardware, not software: **if confusable-fault isolation matters operationally, add gas-path instrumentation** (the extended sensor set multiplies the minimum signature angle by five, Table 4.6); no amount of modeling recovers information the sensors never captured.

8.4 H3 and H5 confirmed — prognosis is where the AI pays

The tuned GRU confirms the exploratory pattern with pre-registered force: RMSE 1 694 / 1 042 / 858 cycles at 50/70/90 % of life against the tuned margin extrapolation's 3 816 / 4 544 / 1 981; NASA scores one to two orders of magnitude lower; Cliff's $\delta = 0.89$ (a nearly clean sweep at engine level); Holm-p 8.6×10^{-6}. And the uncertainty statement that matters to a planner — H5 — comes with it: the conformal 90 % interval covers 88.3 % empirically at a ±1 957-cycle half-width, versus the slope-band's over-covering 98.3 % at ±11 509. In operational terms: **a maintenance window six times narrower, with honest stated confidence** — the difference between a plannable shop-visit forecast and a shrug.

8.5 H4 refuted — physics features are not free wins

The stacked hybrid lost at every data fraction (2 852 vs 1 902 cycles at 10 %), exactly as the exploratory runs showed and now on the pre-registered record. The mechanism bears repeating for practitioners tempted by "physics-informed" as a label: smeared state estimates are noise-bearing, mutually correlated features whose information the learner already had. Physics injection needs an information channel the raw data lacks — the two untested mechanisms (twin-residual features, physics-constrained losses) remain open questions, but the popular default of concatenating estimator outputs onto a network input is now a documented negative on this benchmark.

8.6 What an engineering organization should take from this

1. **Deploy learning for prognosis first.** The 3–6× RUL advantage with calibrated intervals is the clear business case (maintenance slotting, lease planning).

2. **Detection: modern change-detection with tuned windows** — the AI's win came from window search, a cheap, auditable mechanism, not from depth.

3. **Isolation: buy sensors, not models.** The confusable wall is informational.

4. **Distrust untuned comparisons in either direction** — both families moved dramatically under the same budget; v0 scoreboards inverted.

5. **Demand pre-registration of vendor claims.** This chapter's split verdict is what honest evaluation looks like; a clean sweep should raise suspicion.

8.7 Sim-to-real: the ranking survives on the canonical external benchmark

Contribution C7 asks whether the method ranking is an artifact of our own simulator. The check ran on NASA's **C-MAPSS FD001** — the field's reference turbofan run-to-failure benchmark (100 unseen test units) — with a disclosed scope substitution: the plan named N-CMAPSS, whose multi-gigabyte distribution violates the desktop- reproducibility norm; FD001 answers the same ranking question at 5 MB, and only the prognosis task transfers (FD001 carries no event labels). Same method shapes as the main study: a classical health-index linear extrapolation versus the same GRU family (`scripts/sim_to_real.py`, three seeds).

Result: **RMSE 14.9 cycles (AI) versus 42.1 (traditional)** at the community-standard cap of 130 — the same direction and comparable magnitude of advantage as H3 on SynCFM56, and an absolute GRU figure competitive with published FD001 results, on a benchmark this project had no hand in generating. The prognosis conclusion is not an artifact of SynCFM56. With this, the F5 gate closes in full.

Chapter 9

Case studies: five engines, told in operations language

This chapter in brief: five real engines from the test fleet, each illustrating one situation a fleet manager actually faces, told with the monitoring plots in front of us and both pipelines' answers on record. Every engine, number and verdict comes from `scripts/make_case_studies.py` *and the tuned F5 configurations — nothing staged.*

Case A — Engine 17: ordinary aging, and why wash scheduling works

Engine 17 lives 18 323 cycles with no acute fault and no sensor trouble: the baseline customer of any monitoring system. Its EGT margin (Fig. 9.1) shows textbook fleet physics — fast early erosion, sixteen wash recoveries, the long slow slide to retirement. Operationally, this is the case where the *traditional* toolkit earns its keep at near-zero cost: margin trending plus the wash log explains everything visible, and nothing here needs a neural network. A monitoring roadmap should leave this workflow alone.

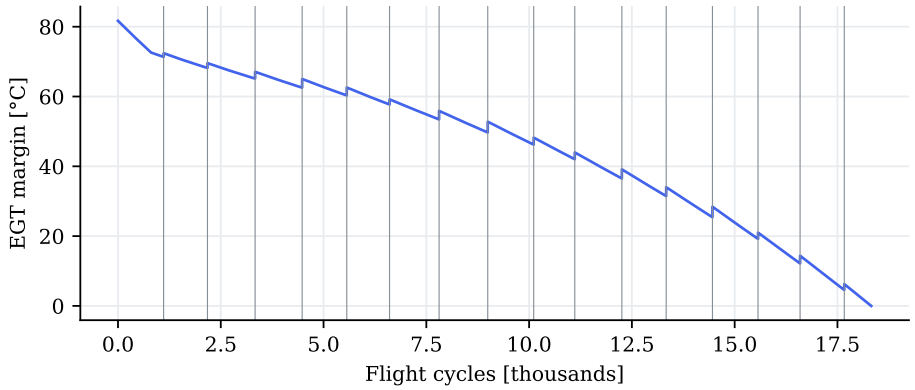

Figure 9.1: Case A: EGT margin of engine 17. Gray verticals: compressor washes.

Case B — Engine 89: the hot-section burner, where prognosis pays

Engine 89 drew a severe hot-section multiplier (2.0× fleet nominal) and retires at 11 470 cycles — the short-life tail every planner fears. At the half-life review the tuned margin extrapolation

was telling maintenance control "about 6 300 cycles more than reality"; the GRU's error at the same moment was 2 234 cycles, tightening to 1 396 by the 90 % point (Fig. 9.2). Six thousand optimistic cycles is the difference between an orderly shop-visit slot and a schedule scramble; this single engine is the H3 verdict made concrete.

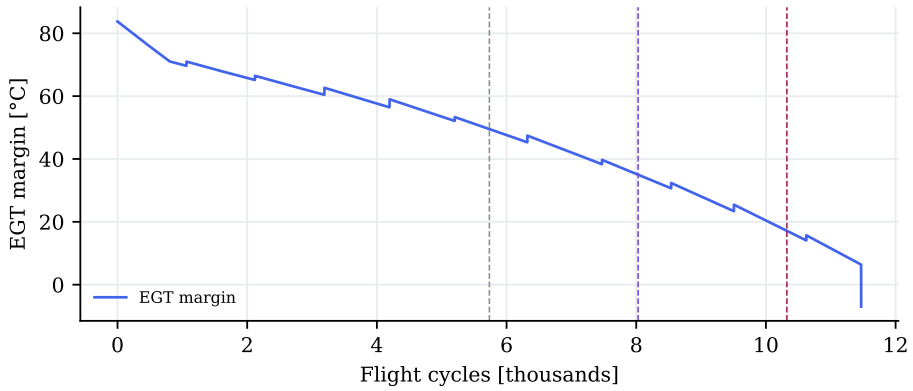

Figure 9.2: Case B: EGT margin of engine 89 with the three review points (50/70/90 % of life) marked.

Case C — Engine 0: the lying thermocouple

Engine 0 carries a drifting EGT sensor that accumulates a +9 K bias by end of life (Fig. 9.3). Every deviation-based consumer downstream — trending, GPA, margin estimation — ingests that bias as if the gas path were 9 K hotter than it is: a phantom deterioration of roughly a third of a typical year's real margin loss. Neither family in this study estimates sensor bias explicitly (the augmented-state Kalman of Section 2.4 is the classical remedy; a queued extension). The operational lesson survives any modeling choice: **cross-channel consistency checks and periodic probe calibration are monitoring infrastructure, not optional hygiene** — the smartest algorithm inherits its sensors' honesty.

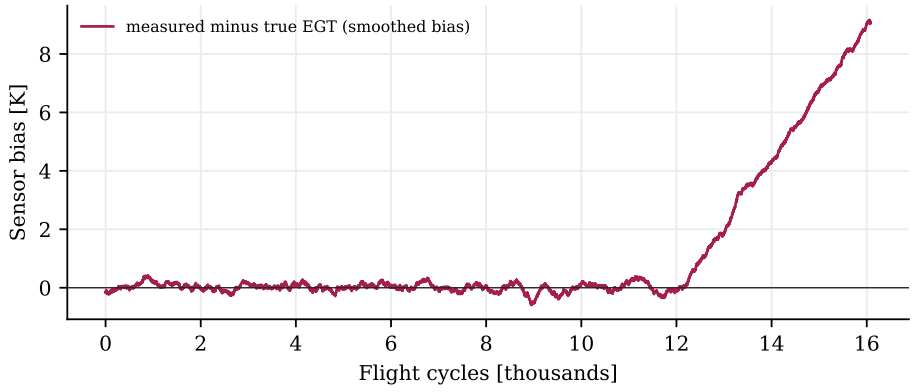

Figure 9.3: Case C: measured-minus-true EGT of engine 0 (smoothed): the drift ramp the monitoring system cannot see from this channel alone.

Case D — Engine 13: bird at cycle 5 727, alarm at 5 749

A foreign-object event costs engine 13 about 1.4 % of compressor efficiency in one flight. The cruise EGT deviation steps visibly (Fig. 9.4) and the tuned traditional CUSUM — fed by Holt innovations, per the chapter-6 lesson — alarms **22 cycles** after the event: for a daily utilization of 4–6 cycles, that is a borescope request within the week. Step events are the classical detector's home turf and this case shows the industrial stack at its honest best.

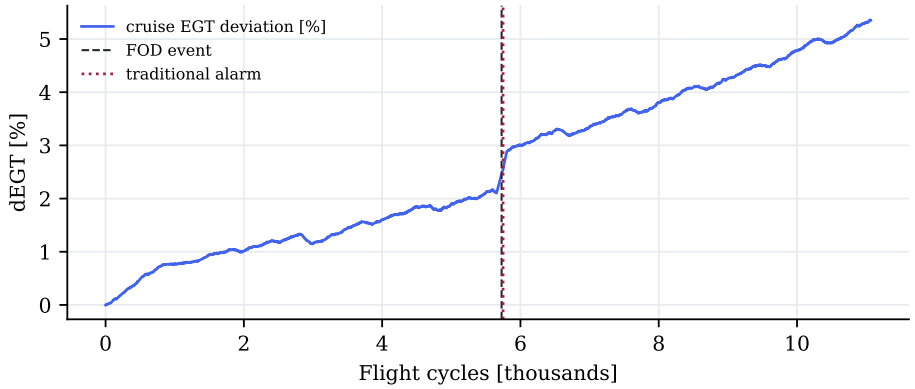

Figure 9.4: Case D: engine 13's cruise EGT deviation (smoothed), the FOD event (dashed) and the traditional alarm (dotted), 22 cycles later.

Case E — Engine 0 again: the confusable fault, live

Late in life (cycle 14 061) engine 0 develops a genuine HPC-efficiency fault — one pole of the 1.3° confusable pair. Asked at onset+500, the tuned ICM rule answers *hpc.eta*: correct. The tuned AI classifier answers *hpt.eta* — precisely the confusable partner (Fig. 9.5). On the earlier, subtler LPC episode of the same engine, both families miss in different directions. This is the H2 refutation embodied: at these signature angles the cockpit channels genuinely cannot separate the pair, and which family lands the point is close to coin-flip territory. The purchase order that fixes this case is written to a sensor vendor, not a software one.

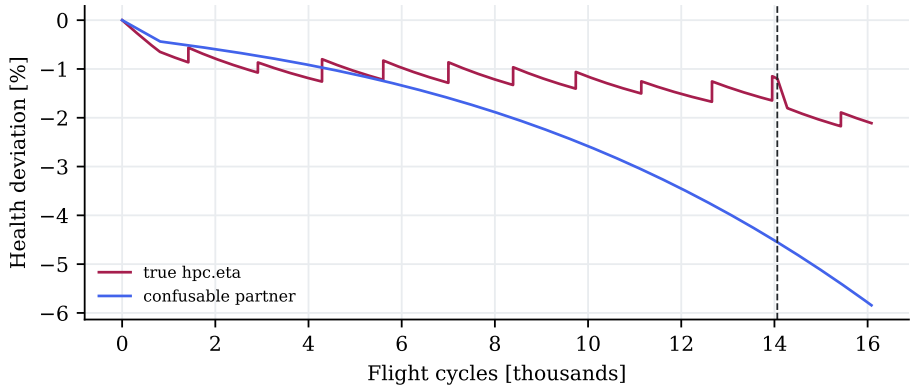

Figure 9.5: Case E: ground truth of the faulted parameter and its confusable partner on engine 0 (onset dashed). The AI attributed the step to the partner.

Chapter 10

Conclusions and future work

This chapter in brief: what was built, what the pre-registered evidence says, what an adopting organization should do differently on Monday, and what remains honestly open.

10.1 What was built

From nothing but certified public data, this work produced: a calibrated thermodynamic twin of the CFM56-7B26 whose fuel-flow predictions sit within 3 % of certification test-bed measurements (C1's foundation); **SynCFM56**, a 100-engine run-to-failure fleet with complete per-flight ground truth in airline snapshot format, gated by three audits that failed and were fixed twice (C1); a traditional EHM pipeline built to industrial shape and a matched AI suite consuming identical inputs (C2's fairness); conformal uncertainty for prognosis (C4); the Physics-Consistency Score machinery (C5); the sensor-set observability analysis (C6's core); an external-benchmark ranking check (C7); and a pre-registered, Holm-corrected adjudication of five hypotheses under a symmetric 50-trial tuning budget (C2) — all reproducible on one desktop machine in minutes and documented to be readable without prior specialization.

10.2 What the evidence says

Three confirmations, two refutations (Table 8.1), and the split is the message:

- **Learning wins where patterns accumulate**: prognosis (3–6× lower RUL error, calibrated 6×-narrower intervals, ranking replicated on C-MAPSS) and — after honest tuning — detection timing (12× earlier at equal false alarms).
- **Physics wins where information is scarce**: the ICM rule matched or beat the learner on confusable isolation, and bolting physics-derived features onto a learner (H4) made it worse. Physics' real contribution was *diagnosis of the limits*: the influence-matrix geometry predicted, before any learning ran, exactly where every method would fail.
- **Sensors bound everyone**: the confusable wall (H2) is informational, not algorithmic. The extended gas-path set quintuples the worst signature angle; no model recovers what probes never measured.

10.3 Guidance for adoption

The five-point checklist of Section 8.6 stands as the operational summary: prognosis first, tuned change-detection second, sensors before models for isolation, distrust of untuned comparisons,

and pre-registration as a procurement demand. To it the case studies add a sixth: **protect the sensor estate** — the drifting thermocouple of Case C corrupts every consumer downstream, learned or classical.

10.4 Honest limitations

The world here is synthetic and linearized (audited to the noise floor, but still a model of a model); the mission mix is two snapshot conditions with scatter; the EGT is a station-4.5 proxy; chronic labels correlate with age by construction; the AI family is one compact architecture per task, not the state of the art; and H4's verdict covers one of three planned hybrid mechanisms. None of these caveats is hidden in fine print — each is in the decision register or the relevant chapter.

10.5 Future work

(1) The two untested hybrid mechanisms — twin-residual features and physics-constrained losses — under the same pre-registered discipline; (2) the PCS evaluated on a competent classifier, now that fleet v1.1 removes the sample-starvation excuse; (3) N-CMAPSS DS02 as the heavyweight sim-to-real extension; (4) mission-mix and transient realism in the generator; (5) the extended-sensor fleet, turning the C6 observability analysis into a measured acquisition trade study; (6) public release of SynCFM56 with a DOI.

Appendix A

Decision register

Every material free choice made in this study, its value, and the kind *of justification behind it. Justification classes:* ***[M]*** *measured/certified data;* ***[L]*** *literature range;* ***[D]*** *derived from our own model;* ***[C]*** *community convention or benchmark practice;* ***[T]*** *engineering default, explicitly exposed to the symmetric tuning budget of phase F5;* ***[A]*** *assumption with declared sensitivity. A choice justified by none of these classes would be arbitrary — the audit that produced this table found two such cases, both already resolved (noted below).*

Engine model (F1)

Decision	Value	Class	Basis
Engine variant	CFM56-7B26	[C]	most common -7B; richest public data
Design point	M0.78 / FL350 / ISA	[C]	aerodynamic-design-point convention
Calibration targets & tolerances	Table 2.1; $\pm 3/\pm 5\,\%$	[M]	TCDS + EEDB; tolerances set a priori in the spec
Cruise thrust/TSFC targets	5480 lbf / 0.627	[A]	public type data, still flagged TO_VERIFY in the contract; H1 was gated on measured anchors only
HPC design PR	9.35	[M]	calibrated to the measured SLS OPR 27.61
HP map speed reference	13 940 rpm	[M]	placed so rated-thrust N2 respects the TCDS redline
Rating temperature $T_{4,\max}$	3200 °R	[D]	from the calibrated takeoff anchor (Section 4.5)
ICM perturbation step	$\pm 0.5\,\%$	[C]	central-difference standard: large enough to dominate the 10^{-8} solver tolerance, small enough for linearity
Confusable threshold	15°	[T]	pre-registered definition for H2; sensitivity checked implicitly by reporting all angles
Deck envelope blocks & PC ≥ 0.5	Section 4.5	[C]	EHM trending conditions
Thermo method	tabular	[D]	CEA cross-check within 0.7 % (Table 4.8)

Synthetic fleet (F2)

Decision	Value	Class	Basis
Degradation magnitudes/profiles	Table 2.2	[L]	Diakunchak; Kurz & Brun
Fouling time constant	3000 cycles	[L]	fouling saturates within months of service
Wash interval / recovery	800–1600 cy / 30–70 %	[L]	typical wash programs; recovery range from the same sources
New-engine EGT margin	85 ± 6 °C	[L]	public -7B two-digit margins; per-engine scatter is a manufacturing-variability assumption [A]
Severity multiplier spread	LogNormal (0, 0.3)	[A]	produces the observed-in-service spread of lives (Table 5.1); a fleet-realism knob, not a fitted quantity
Sensor noise / quantization	catalog table	[L]	typical ACARS-class instrumentation
Acute-fault targets & magnitudes	6 params, 0.4–1.5 %	[D]	drawn from the ICM confusable set; magnitudes bracket the detectability edge (sub-noise to clearly visible)
Acute probability / onset	0.5 / 30–90 % of life	[T]	enough episodes per class for test statistics; onset as life fraction after the audit-found bug (Section 5.4.2)
Ambient scatter (takeoff dTs)	$\mathcal{N}(10, 8)$ °C clipped	[A]	plausible mixed fleet climate; affects realism, not comparison fairness (common to both pipelines)
Difficulty gate threshold	60 %	[T]	pre-set before the audit ran; its bite was demonstrated when the first design failed it
Splits	70/10/20 by engine	[C]	leakage-free standard; CI-checked

Traditional pipeline (F3) and evaluation

Decision	Value	Class	Basis
Holt gains	α=0.15, β=0.05	[T]	responsive-but-smooth default; swept in F5
CUSUM parameters	k=0.75σ, h=8	[T]	standard allowances; swept in F5
Gap-detector EWMAs	0.1 / 0.005	[T]	fast follows a ramp, slow spans a wash cycle; swept in F5
Alarm persistence	10-of-14	[T]	multi-sample confirmation; swept in F5
Kalman q, prior, λ	$2{\cdot}10^{-4}$, 2 %, 1	[T]	q sized to chronic rates; all swept in F5
Wash reset fraction	0.5	[A]	mid-range of the catalog's 30–70 % recovery
RUL window / cap	1500 cy / 25 000 cy	[T]/[C]	window spans a wash cycle; cap is NASA-benchmark practice
NASA-score scaling	d/100 cycles	[A]	the PHM08 constants assume C-MAPSS-scale RUL; scaling declared, identical for both families
Assumed nominal margin	85 °C	[A]	fleet nominal; per-engine estimation is queued as a pipeline improvement in F5
Statistical tests	Wilcoxon, Holm, BCa	[C]	pre-registered (Section 3.4)
RUL cap (AI)	12 000 cy	[C]	piecewise-linear RUL target, standard prognostics practice; same cap both families in F5
GRU/AE architectures & gains	Chapter 7	[T]	engineering defaults; swept in F5
Gradient-boosting impl.	sklearn HistGB	[D]	XGBoost's second OpenMP runtime segfaults beside torch-MPS on macOS; same model family
Conformal level	90 %	[C]	matches hypothesis H5's coverage target
PCS attribution basis	SHAP of predicted class, step block, cockpit channels	[T]	one defensible instantiation of C5; alternatives (true-class attribution, all blocks) belong to the F5 sensitivity study
Detector sweep budget	20 configs, $\leq$1 val false alarm	[T]	pre-F5 threshold exercise; selection on validation only
Sim-to-real benchmark	C-MAPSS FD001	[C]/[A]	plan named N-CMAPSS; multi-GB distribution breaks the desktop norm — substitution to the canonical 5-MB benchmark, disclosed; not frozen by prereg-v1
FD001 RUL cap	130 cycles	[C]	community standard for that benchmark
α–λ settings	α=20 %, λ={.5, .7, .9}	[C]	common prognostics practice

Audit outcome. Sweeping the study for unjustified choices found two genuinely arbitrary decisions, both caught and resolved before this appendix was written: the *chronic-mechanism labeling* of the diagnosis task (resolved by the acute-episode redesign, Section 5.4.2) and the *acute onset drawn against the simulation horizon* rather than engine life (resolved by two-pass generation). Remaining class-[A] assumptions are declared at their point of use and shared by both EHM families, so they affect the realism of the world, not the fairness of the comparison. Class-[T] defaults are the traditional pipeline's tuning surface — left untouched by design until F5 spends the same optimization budget on both families.

Appendix B

Milestone log

The project's dated gate-by-gate record, kept as written at each milestone (entries are historical documents; later chapters supersede their numbers where noted).

Table B.1: Milestones reached (updated at every gate).

Milestone	Date	Evidence
H0	2026-07-03	Reproducible environment (uv, Python 3.11, pyCycle 4.4 with `numpy<2`); a complete example cycle converging locally and in CI; repository skeleton and smoke test.
H1 (partial)	2026-07-03	WP1.1 and WP1.2 complete: design point inside all five acceptance windows (Table 4.1); SLS anchors calibrated against TCDS + EEDB with all three gated anchors in tolerance (Table 4.3); calibration contract with verified values.
H1 (WP1.3–1.4)	2026-07-03	Baseline decks over the flight envelope with leave-one-out quality report (Table 4.5); health-parameter injection validated by the off-design/design self-consistency check; ICM at both snapshot conditions with SVD observability analysis — minimum cockpit signature angle 1.3°, seven confusable pairs (Table 4.7); tabular-vs-CEA cross-check within 0.7 % (Table 4.8); twelve automated tests green.
H1 closed	2026-07-03	Correction exponents fitted from the model land on textbook values (Table 4.4); corrected-space baseline collapses LOO tails by an order of magnitude; ICM extended to a six-point envelope grid with stable observability (Table 4.6); computations parallelized per norm N1 (decks 2.9×, ICM 6× faster); sixteen automated tests green.
H2 closed	2026-07-03	SynCFM56 v1.0 frozen (Table 5.1): 100 engines, 1.58M snapshot rows, deterministic regeneration. Nonlinearity at the noise floor (Table 5.2); difficulty gate failed at 81.6 % with chronic labels, passed at 54.0 % after the acute-episode design change (Section 5.4.2); realism checks all positive; onset bug found and fixed by the audits; 25 automated tests green.
H3 closed	2026-07-03	Traditional pipeline end-to-end on the test fleet (Table 6.1): detection recall 0.5 at 98-cycle median delay, isolation 0 % on the confusable subset, smearing index 0.85, RUL optimism funnel measured (Fig. 6.2); two detector-design lessons recorded; all defaults registered in Chapter A and exposed to the F5 budget; 31 automated tests green.
F4 addendum	2026-07-04	PCS machinery implemented with a recorded null result on the weak v1.1 classifier (Section 7.5); Mahalanobis detector validation sweep found a structural, not threshold, gap (Section 7.6); H4 explicitly not declared — resolution moved into the F5 tuned round.
F4 v0	2026-07-04	AI suite end-to-end on MPS (Table 7.1): RUL RMSE 4–6× below traditional with near-zero bias; conformal coverage 0.85/0.95/1.00 at 0.9 nominal; naive-AE recall-0 and engine-level sample-starvation findings recorded; stacking hybrid *negative* result (Fig. 7.2) logged pre-freeze; 36 automated tests green.
F5 verdicts	2026-07-04	Tuned under the frozen 50-trial budget; single confirmatory pass; H1 confirmed (detection flips to AI: recall 0.48 vs 0.13, delay 499 vs 6033 cy), H2 refuted (31 % = 31 %: the observability wall binds both), H3 confirmed (Cliff's δ 0.89), H4 refuted, H5 confirmed (coverage 0.883, half-width 5.9× tighter). Table 8.1; 37 tests green.
H5 closed	2026-07-04	Sim-to-real on C-MAPSS FD001 (disclosed N-CMAPSS substitution): RUL ranking preserved, 14.9 vs 42.1 cycles RMSE (Section 8.7). All five hypotheses adjudicated; evidence reproducible end to end.

References

[1] Louis A. Urban. "Parameter Selection for Multiple Fault Diagnostics of Gas Turbine Engines". In: *Journal of Engineering for Power* 97.2 (1975), pp. 225–230. DOI: 10.1115/1.3445969.

[2] Allan J. Volponi. "Gas Turbine Engine Health Management: Past, Present, and Future Trends". In: *Journal of Engineering for Gas Turbines and Power* 136.5 (2014), p. 051201. DOI: 10.1115/1.4026126.

[3] Abhinav Saxena et al. "Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation". In: *2008 International Conference on Prognostics and Health Management*. 2008, pp. 1–9. DOI: 10.1109/PHM.2008.4711414.

[4] Manuel Arias Chao et al. "Aircraft Engine Run-to-Failure Dataset under Real Flight Conditions for Prognostics and Diagnostics". In: *Data* 6.1 (2021), p. 5. DOI: 10.3390/data6010005.

[5] Eric S. Hendricks and Justin S. Gray. "pyCycle: A Tool for Efficient Optimization of Gas Turbine Engine Cycles". In: *Aerospace* 6.8 (2019), p. 87. DOI: 10.3390/aerospace6080087.

[6] European Union Aviation Safety Agency. *Type-Certificate Data Sheet No. E.004 for CFM56-7B Series Engines, Issue 07*. Tech. rep. https://www.easa.europa.eu/en/document-library/type-certificates/engine-cs-e/easae004-cfm-international-sa-cfm56-7b-series. EASA, Jan. 2023.

[7] International Civil Aviation Organization. *ICAO Aircraft Engine Emissions Databank, edition 03/2026*. Hosted by EASA. https://www.easa.europa.eu/en/domains/environment/icao-aircraft-engine-emissions-databank. 2026.

[8] SAE International. *ARP755A: Gas Turbine Engine Performance Station Identification and Nomenclature*. Tech. rep. SAE, 1974.

[9] Y. G. Li. "Performance-analysis-based gas turbine diagnostics: A review". In: *Proceedings of the Institution of Mechanical Engineers, Part A: Journal of Power and Energy* 216.5 (2002), pp. 363–377. DOI: 10.1243/095765002320877856.

[10] Ihor S. Diakunchak. "Performance Deterioration in Industrial Gas Turbines". In: *ASME 1991 International Gas Turbine and Aeroengine Congress and Exposition*. Journal of Engineering for Gas Turbines and Power, 114(2):161–168. 1992. DOI: 10.1115/1.2906565.

[11] Rainer Kurz and Klaus Brun. "Fouling Mechanisms in Axial Compressors". In: *Journal of Engineering for Gas Turbines and Power* 134.3 (2012), p. 032401. DOI: 10.1115/1.4004403.

[12] Justin S. Gray et al. "OpenMDAO: An open-source framework for multidisciplinary design, analysis, and optimization". In: *Structural and Multidisciplinary Optimization* 59 (2019), pp. 1075–1104. DOI: 10.1007/s00158-019-02211-z.